Advanced OO concepts

Chapter 4

V3 Fall 2026-2027

Content

- **Abstraction and encapsulation using Classes**
- **Relationships between Classes**
- **Superclass** and Subclass
- Polymorphism
- Interfaces
- Important Interfaces: The Comparable Interface
- Immutable Classes and Objects
- Cloneable Classes

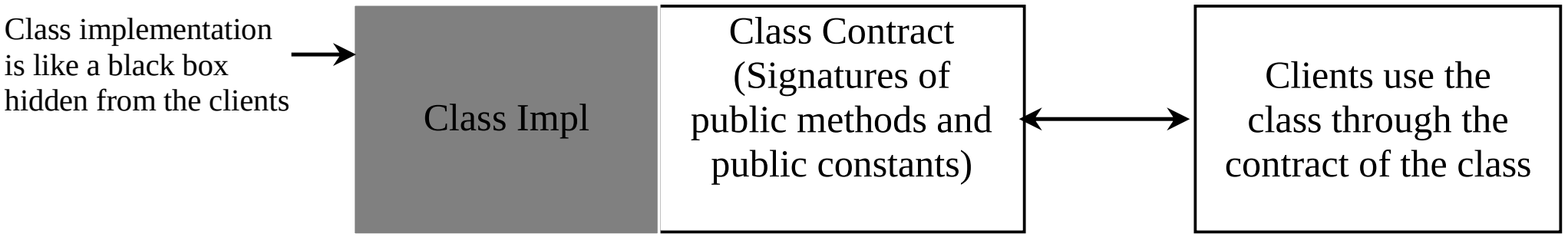
Learning Outcomes

By the end of this unit, you should be able to demonstrate the knowledge of the following and understand their contributions towards improved software quality and innovation.

- Abstraction and encapsulation using Classes
- Relationships between Classes
- Superclass and Subclass
- Polymorphism
- Interfaces
- Important Interfaces: The Comparable Interface
- Cloneable Classes
- Immutable Classes and Objects

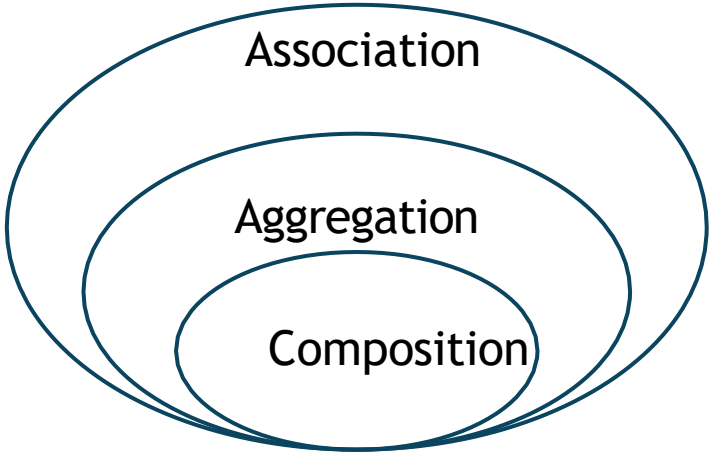
Class Abstraction and Encapsulation

- Class abstraction means to separate class implementation from the use of the class.
- The creator of the class provides a description of the class and let the user know how the class can be used.
- The user of the class does not need to know how the class is implemented.
- The detail of implementation is encapsulated and hidden from the user.



Class Relationships

- In object-oriented programming, relationships between classes play a crucial role in defining how objects interact with each other.
- Java, being an object-oriented language, provides mechanisms to model these relationships through association, aggregation, and composition.
- Aggregation, association, and composition in Java describe how instances of classes relate to each other.
- Four types of relationship between classes are possible
  - Association
  - Aggregation
  - Composition
  - Inheritance



Class Relationships

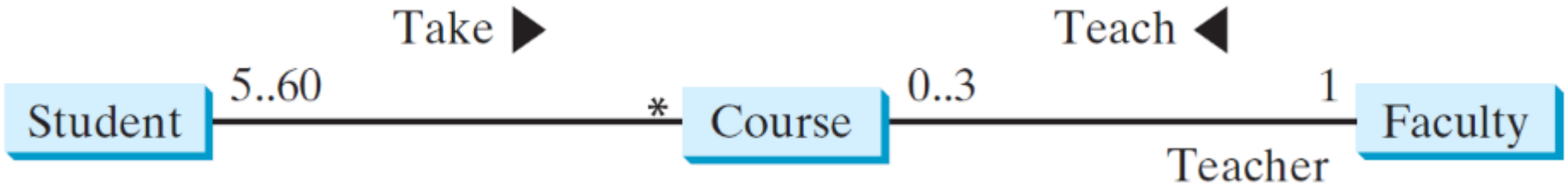
Association

Aggregation

Composition

Inheritance

Association: is a general binary relationship that describes an activity between two classes.



Association

- Association can be considered a generic term to indicate the relationship between two independent classes; the relationship may be one-to-one, one-to-many, or many-to-many, but it need not indicate ownership.

Association: is a general binary relationship that describes an activity between two classes.

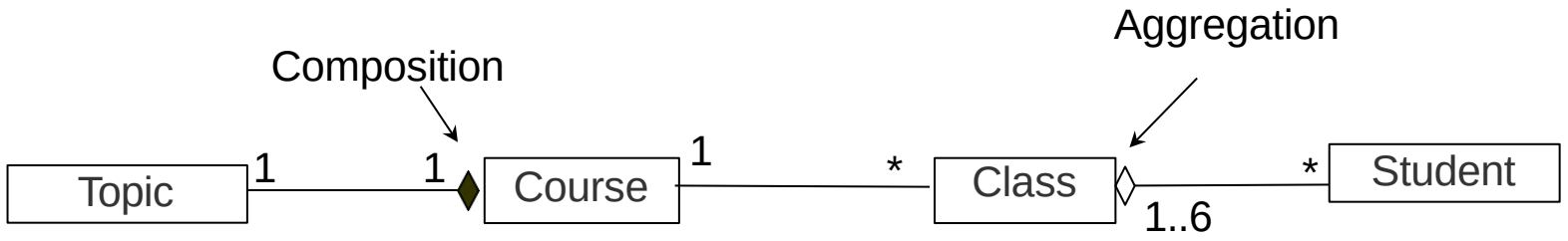


Aggregation

- Aggregation is a specific form of association in which one class, *the whole*, contains a collection of other classes, *the parts*; here, however, the lifecycle of the parts does not depend upon the whole. For example, a library and books show aggregation, since books may exist somewhere apart from the library.
- In contrast, composition is a stronger form of aggregation that means ownership and lifecycle dependence; if *the whole* gets destroyed, then *the parts* no longer exist. For composition, a good example would be a house and its different rooms; a room cannot exist without a house.

Aggregation - Object Composition

- Composition is actually a special case of the aggregation relationship.
- Aggregation models the *has-a* relationships and represents an ownership relationship between two objects.
- The owner object is called an aggregating object and its class an aggregating class.
- The subject object is called an aggregated object and its class an aggregated class.



Class Relationships Representation

- An aggregation relationship is usually represented as a data field in the aggregating class.
- For example, the relationships in the previous Figure can be represented as follows:

```
public class Topic
{

  …



}
```

```
public class Course {
   private Topic[] myTopics;

  …



}
```

```
public class Class {
   private Course course;
  private Student[] students;

  …



}
```

```
public class Student {
   private Class[]
myClasses;

  …

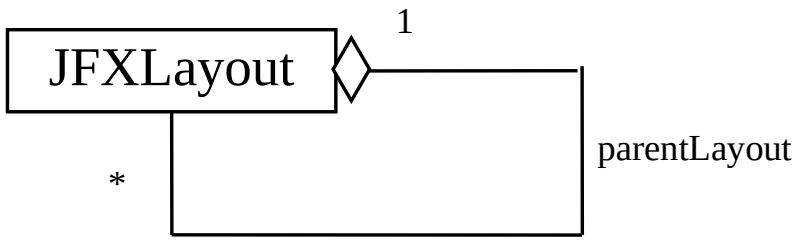


}
```

Aggregation Between Same Class

Aggregation may exist between objects of the same class.

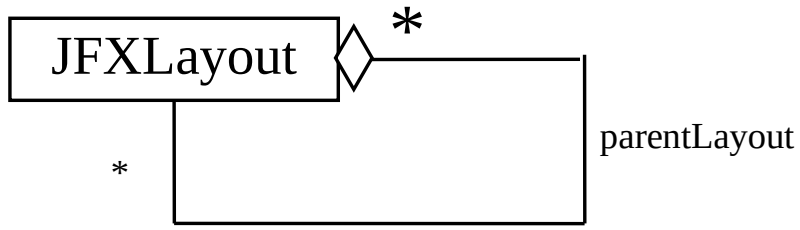
For example, a JFXLayout may have a parent JFXLayout.




```
public class JFXLayout {
    // The type for the data is the class itself
    private JFXLayout parentLayout;
    private List<JFXLayout> children;
    ...
}
```

Aggregation Between Same Class

What happens if a JFXLayout can be displayed inside many parent layouts?



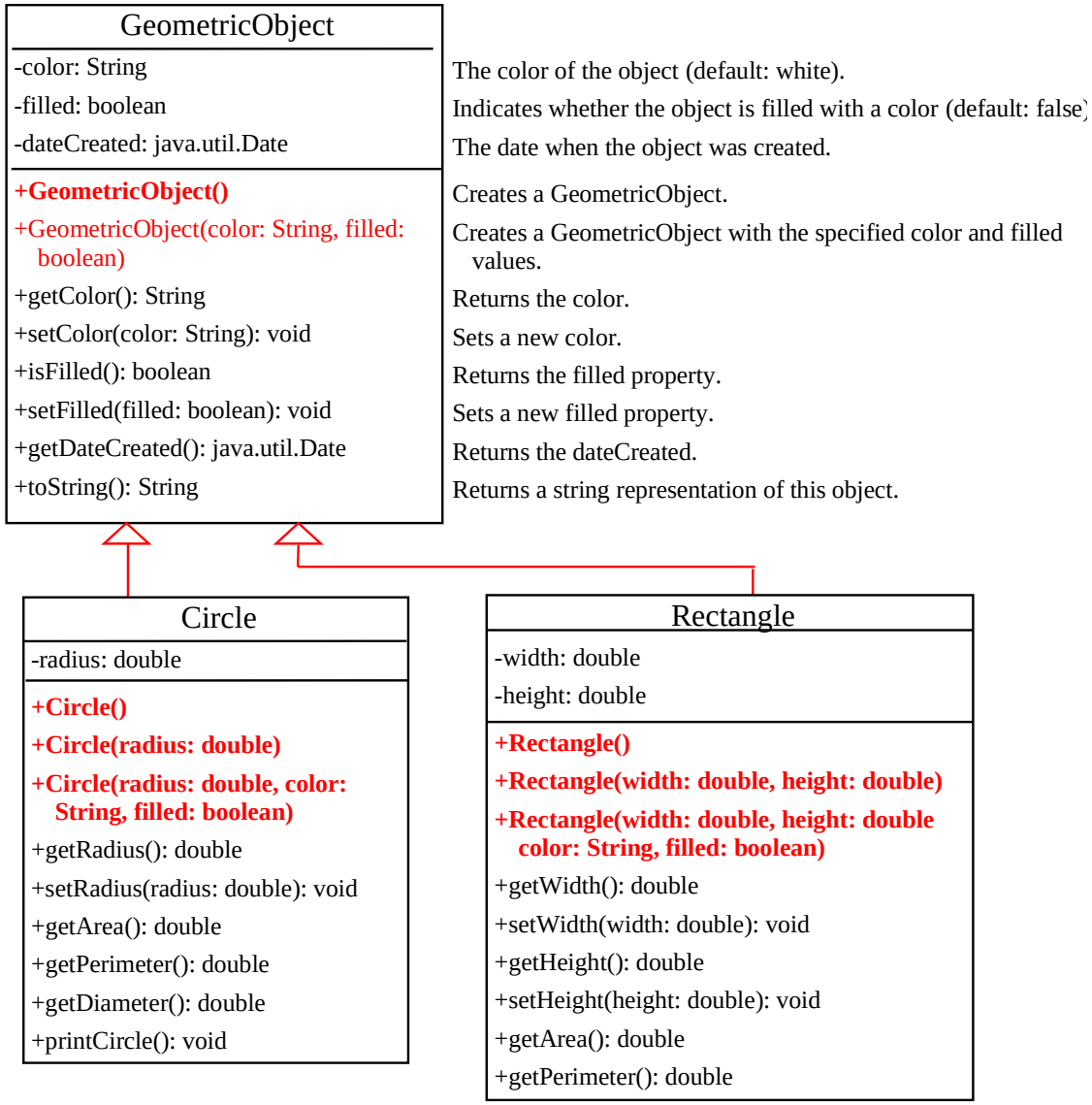

```
public class JFXLayout {
   // The type for the data is the class itself
   private JFXLayout[] parentLayouts;
  ...
}
```

```
// main class
public class App {
   // The reusable layouts are also stored
   //  in a more general variable
   private JFXLayout[] reusableLayouts;
  ...
}
```

Inheritance

- Inheritance represents *is-a* relationship in Java. It establishes a relationship between a more general class (known as superclass) and a more specialized class (known as subclass).
- In other words, *is-a* relationship defines the relationship between two classes in which one class extends another class.
- For example, if a class B inherits from class A, their relationship will by the keyword “extends” and B can inherit data members from class A.

Superclasses and Subclasses constructors



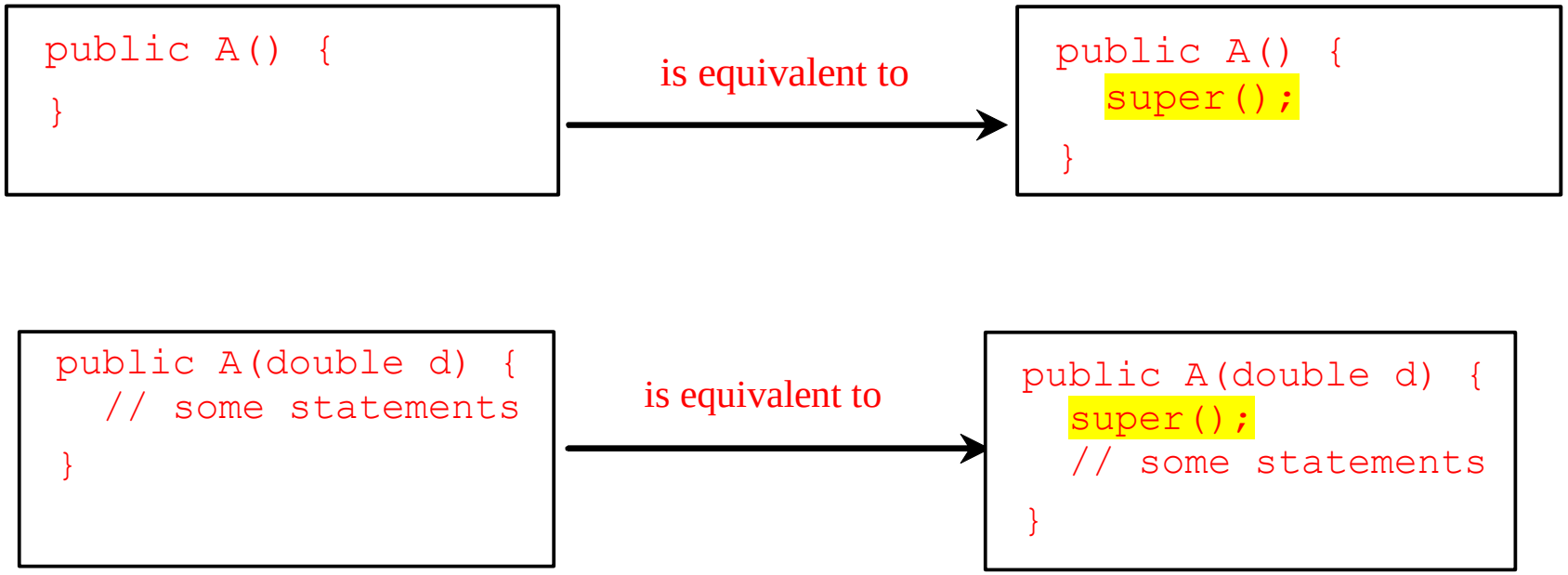
Let us consider the following example

Are superclass' Constructor Inherited?

- A constructor is used to construct an instance of a class.
- Unlike properties and methods, a superclass's constructors are not inherited in the subclass.
- They can only be *explicitly* invoked from the subclasses' constructors, using the keyword **`super`**.
- If the keyword super is not explicitly used, the superclass's no-arg constructor is automatically invoked (implicitly).

Superclass's Constructor Is Always Invoked

A constructor may invoke an overloaded constructor or its superclass's constructor. If none of them is invoked explicitly, the compiler puts super() as the first statement in the constructor. For example,



Using the Keyword `super`

- The keyword super refers to the superclass of the class in which super appears. This keyword can be used in two ways:
  - To call a superclass constructor
  - To call a superclass method (when the method is overridden in the subclass)

CAUTION

You must use the keyword super to call the superclass constructor.

Invoking a superclass constructor's name in a subclass causes a syntax error.

Java requires that the statement that uses the keyword **super appear first** in the constructor.

Constructor Chaining

Constructing an instance of a class invokes all the superclasses' constructors along the inheritance chain. This is known as *constructor chaining*.

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

1. Start from the main method

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

2. Invoke Faculty constructor

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

3. Invoke Employee's no-arg constructor

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

4. Invoke Employee(String) constructor

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

5. Invoke Person() constructor

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

6. Execute println

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

7. Execute println

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

8. Execute println

animation

Trace Execution

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }

  public Employee(String s) {
    System.out.println(s);
  }
}

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

9. Execute println

Output

(1) Person's no-arg constructor is invoked
(2) Invoke Employee's overloaded constructor
(3) Employee's no-arg constructor is invoked
(4) Faculty's no-arg constructor is invoked

Example on the Impact of a Superclass without no-arg Constructor

Find out the errors in the program:

```
public class Apple extends Fruit {
}

class Fruit {
  public Fruit(String name) {
    System.out.println("Fruit's constructor is invoked");
  }
}
```

Problem:

```
public class Apple extends Fruit {
    public Apple() {
    }
}
```

Implicit super constructor Fruit() is undefined. Must explicitly invoke another constructor

Press 'F2' for focus

A Solution:

```
public class Apple extends Fruit {
    public Apple() {
        super("apple");
    }
}
```

Exercise

```
public class Car extends Vehicle {
        public Car() {
                System.out.println("Car constructor");
                }
                        }
class Vehicle {
        public Vehicle(String brand) {
                System.out.println("Vehicle constructor");
                }
                        }
```

Exercise

- Will the program compile? Why?
- Which constructor is automatically called from the Car constructor?
- Identify the error in the program.
- **Fix the program in two different ways.**
- What will be printed when you create the following object using the original program? Car car = new Car();
- For each corrected version, what will be the output of: Car car = new Car();

Solution

- **Will it compile?** No, because Car implicitly calls super(), but Vehicle has no no-argument constructor.
- **Which constructor is automatically called?** Vehicle() through the implicit super() call.
- **What is the error?** Vehicle() is undefined because only Vehicle(String brand) exists
  - **Fix 1:** Add super("Toyota"); inside the Car constructor.
  - **Fix 2:** Add a no-argument Vehicle() constructor.
- **Original output:** Nothing, because the program does not compile.
- **Fix 1 output:** Vehicle constructor → Car constructor.
- **Fix 2 output:** Vehicle constructor → Car constructor.

Designing a Class

Coherence

- A class should describe a single entity, and all the class operations should logically fit together to support a coherent purpose.
- You can use a class for students, for example, but you should not combine students and staff in the same class, because students and staff have different entities.

Designing a Class

Separating responsibilities: Single Responsibility Pattern SOP

- A single entity with too many responsibilities can be broken into several classes to separate responsibilities.
- The classes String, StringBuilder, and StringBuffer all deal with strings, for example, but have different responsibilities.
- The String class deals with immutable strings, the StringBuilder class is for creating mutable strings, and the StringBuffer class is similar to StringBuilder except that StringBuffer contains synchronized methods for updating strings.

Designing a Class

Design for reuse.

- Users can incorporate classes in many **different combinations**, orders, and environments.
- Therefore, you should design a class that **imposes no restrictions** on what or when the user can do with it, design the properties to ensure that the user can set properties in any order, with any combination of values, and design methods to function independently of their order of occurrence.
- Provide a public **no-arg constructor** and **override the equals** method and the **toString** method defined in the Object class whenever possible.

Designing a Class

Follow standard programming style and naming conventions e.g.

- Choose **informative names** for classes, data fields, and methods.
- Always **place** the data declaration before the constructor, and place constructors before methods.
- Always provide a **constructor and initialize** variables to avoid programming errors.

Using Visibility Modifiers

- Each class can present two contracts – one for the users of the class and one for the extenders of the class.
- Make the **fields private** and **accessor methods public** if they are intended for the users of the class.
- Make the **fields or method protected** if they are intended for extenders of the class.
- The contract for the extenders encompasses the contract for the users.

Defining a Subclass

- A subclass inherits from a superclass. You can also:
  - Add new properties
  - Add new methods
  - Override the methods of the superclass

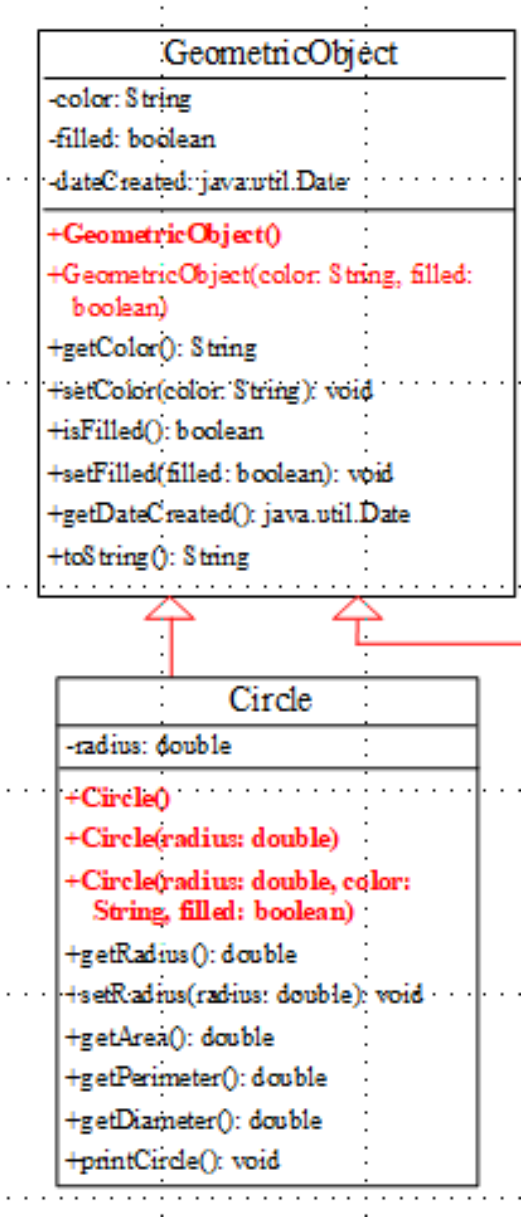
Calling Superclass Methods

You could rewrite the printCircle() method in the Circle class as follows:

```
public void printCircle() {

  System.out.println("The circle is created " +
    super.getDateCreated() + " and the radius is " + radius);

}
```



Overriding Methods in the Superclass

- A subclass inherits methods from a superclass.
- Sometimes it is necessary for the subclass to modify the implementation of a method defined in the superclass. This is referred to as *method* ***overriding***.

```
public class Circle extends GeometricObject {
  // Other methods are omitted


  /** Override the toString method defined in GeometricObject */
  public String toString() {
    return super.toString() + "\nradius is " + radius;
  }
}
```

NOTE

An instance method can be overridden only if it is accessible. Thus a **private method cannot be overridden**, because it is not accessible outside its own class. If a method defined in a subclass is private in its superclass, the two methods are completely **unrelated**.

```
public class Fruit {

      public Fruit(String name) {

      private void eat() {

      }

}
```

```
public class Apple extends Fruit {
    public Apple() {

    @Override
    public void eat() {

    }
}
```

The method eat() of type Apple must override or implement a supertype method

1 quick fix available:

Remove '@Override' annotation

If you change the visibility to protected or public, the error is solved

NOTE

Like an instance method, a static method can be inherited. However, **a static method cannot be overridden**. If a static method defined in the superclass is redefined in a subclass, the method defined in the superclass is hidden:

At runtime if the variable (not the object referenced by that variable) has the type of the superclass, the superclass method is invoked, and if it has the subclass variable, the subclass method will be invoked: no runtime polymorphism

Example: Static method

```
class Person {
    public static void display() {
        System.out.println("Person");
    }
}

class Student extends Person {
    public static void display() {
        System.out.println("Student");
    }
}

Person p = new Student();
p.display();// is same as Person.display();
Student s = new Student();
s.display();
```

Static methods should normally be accessed using the class name, so we don't need to create an object just to call display():We created the objects **only to demonstrate the important difference between static methods and instance methods**.

Instance method → object determines the method → runtime polymorphism
Static method → variable/class type determines the method → no runtime polymorphism

Overriding vs. Overloading

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // This method overrides the method in B
  public void p(double i) {
    System.out.println(i);
  }
}
```

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // This method overloads the method in B
  public void p(int i) {
    System.out.println(i);
  }
}
```

Polymorphism

```
public class PolymorphismDemo{
  public static void main(String[] args) {
    m(new GraduateStudent());
    m(new Student());
    m(new Person());
    m(new Object());
  }

  public static void m(Object x){
    System.out.println(x.toString());
  }
}


class Person extends Object{
  public String toString(){
    return "Person";
  }
}


class Student extends Person{
  public String toString(){
    return "Student";
  }
}


class GraduateStudent extends Student{
}
```

Polymorphism means that a variable of a supertype can refer to an object of the subtype

In other words, an object of a subtype can be used wherever its supertype value is required.

For example, method m takes a parameter of the Object type. You can therefore invoke it with any object.

x may be an instance of GraduateStudent, Student, Person, or Object.

When the method `m(Object x)` is executed, the argument `x's toString` method is invoked.

Classes Student, Person, and Object have their own implementation of the `toString` method.

Which implementation is used will be determined dynamically by the Java Virtual Machine at runtime. This capability is known as *dynamic binding*.

Dynamic Binding

- Suppose an object o is an instance of classes C1, C2, ..., Cn-1, and Cn, where C1 is a subclass of C2, C2 is a subclass of C3, ..., and Cn-1 is a subclass of Cn i.e. Cn is the most general class, and C1 is the most specific class.



- If object o invokes a method p, the JVM searches the implementation for the method p in C1, C2, ..., Cn-1 and Cn, in this order, until it is found
- Once an implementation is found, the search stops and the first found implementation is invoked.

Example

```
class C3 {
    void p() {
        System.out.println("C3");
    }
}

class C2 extends C3 {
    void p() {
        System.out.println("C2");
    }
}

class C1 extends C2 {
    void p() {
        System.out.println("C1");
    }
}
```

C3 x = new C1(); x.p();

C1: Does it have p()? YES → use C1.p()
C2: don't search
C3: don't search

Polymorphism

- Polymorphism is a popular concept in object-oriented programming (OOP), referring to the idea that an entity in code such as a variable, method or **object** can have more than one form.
- Polymorphism is a fundamental concept in object-oriented programming (OOP) that allows **objects** of different types to be accessed through the same interface.
- This means that a single function or method can operate on different kinds of objects, each providing its own implementation.

Polymorphism

- Compile time polymorphism
  - Also known as *static polymorphism*, compile time polymorphism is common in OOP languages like Java.
  - It uses method *overloading* to create multiple methods that have the same name in the same class, but a different number of parameters.
  - They may also have parameters for different data types.
- Runtime polymorphism
  - Also known as *dynamic polymorphism*, runtime polymorphism uses method *overriding* to let a child class have its own definition of a method belonging to the parent class.
  - This type of polymorphism is often associated with upcasting, which happens when a parent class points to an instance of the child's class.

Polymorphism Example

- Create a screen manager Class that can draw objects of different types (classes) stored in a given collection of objects.
- The screen manager draws these objects into a video game screen with minimal modifications to the system's code whenever a new type (class) of objects is created
- How do we solve the problem?

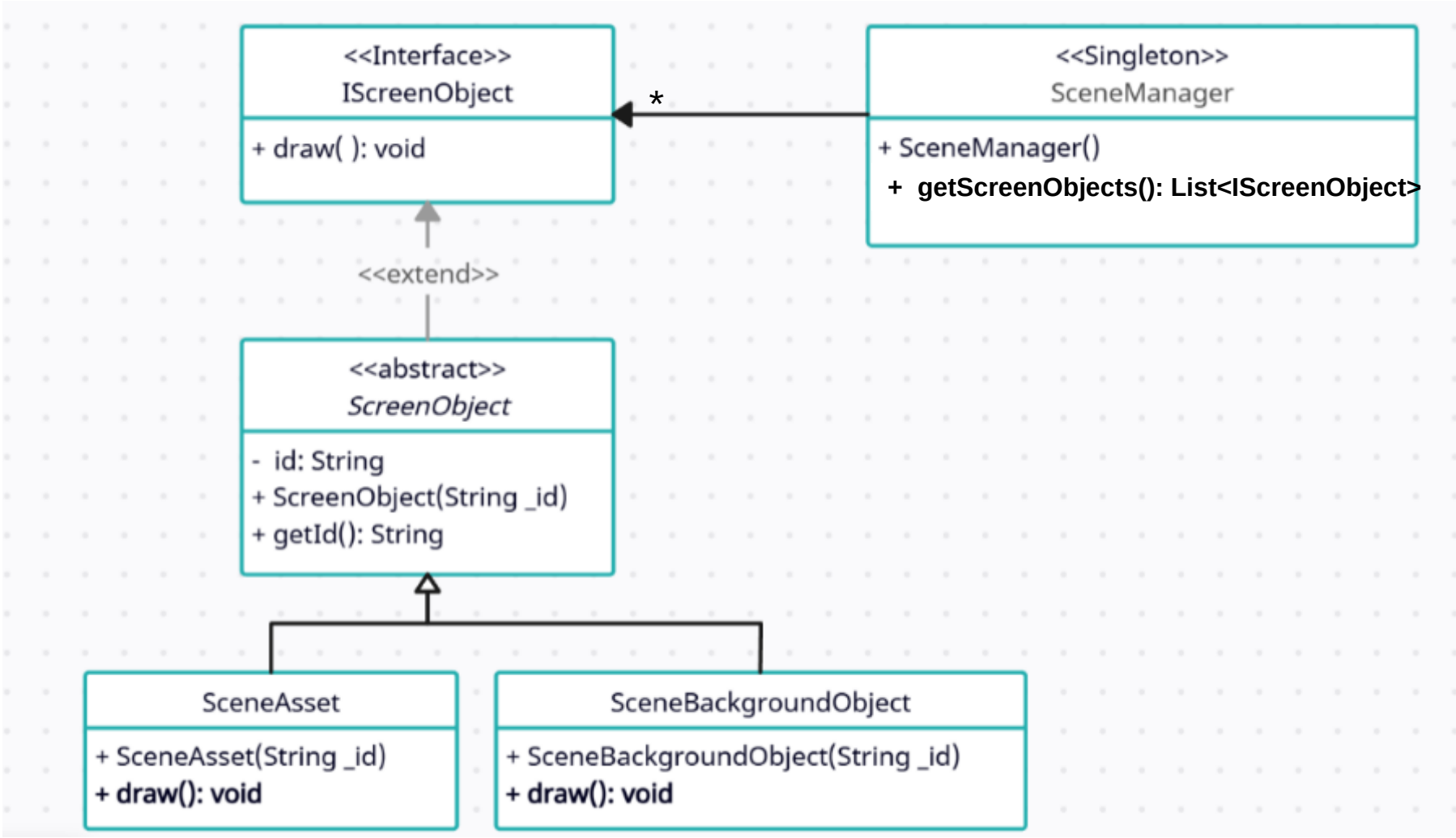
© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Polymorphism Example - Approach

- Create an **interface IScreenObject** that has a draw method.
- Build **scene classes** that implement IScreenObject and provide their own draw method implementation.
- When objects of different classes appear in a IScreenObject collection of objects to draw, the screen-manager code invokes method draw, exactly as it does for every other object in the collection, regardless of its type.
- So the objects simply have the right interface. And when new scene classes are created, no modification to the screen manager is needed.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Polymorphism Review: Exercise (solution)



Source Code: workspace_ch_04/polymorph01



Software Engineering Observation 10.1

Polymorphism enables you to deal in generalities and let the execution-time environment handle the specifics. You can tell objects to behave in manners appropriate to those objects, without knowing their specific types, as long as they belong to the same inheritance hierarchy.



Software Engineering Observation 10.2

Polymorphism promotes extensibility: Software that invokes polymorphic behavior is independent of the object types to which messages are sent. New object types that can respond to existing method calls can be incorporated into a system without modifying the base system. Only client code that instantiates new objects must be modified to accommodate new types.

***Note**: The meaning of "Messages sent to some object" corresponds to an OO vocabulary that means calling methods of that object: if an object A calls someOtherObject.someMethod() -> we say that object A sent the "someMethod" message to someOtherObject.*

Case Study: Payroll System Using Polymorphism

A company pays its employees on a weekly basis. The employees are of four types,

- Salaried employees that are paid a fixed weekly salary regardless of the number of hours worked,
- Hourly employees that are paid by the hour and receive overtime pay (i.e., 1.5 times their hourly salary rate) for all hours worked in excess of 40 hours,
- Commission employees that are paid a percentage of their sales and
- Base-salaried commission employees that receive a base salary plus a percentage of their sales.

For the current pay period, the company has decided to reward base-salaried commission employees by adding 10% to their base salaries.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Case Study: Payroll System Using Polymorphism

- A company requires a Java application that performs its payroll calculations polymorphically.
- Use an abstract method and polymorphism to perform payroll calculations based on the type of inheritance hierarchy headed by an employee.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Case Study: Payroll System Using Polymorphism (Cont.)

- `abstract` class `Employee` represents the general concept of an employee.
- Subclasses: `SalariedEmployee`, `CommissionEmployee` , `HourlyEmployee` and `BasePlusCommissionEmployee` (an indirect subclass)
- Fig. 10.2 shows the inheritance hierarchy for our polymorphic employee-payroll application.
- Abstract class names are italicised in UML.

Recall that an abstract class in Java is a class that cannot be instantiated on its own and must be subclassed by another class to utilize its properties.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

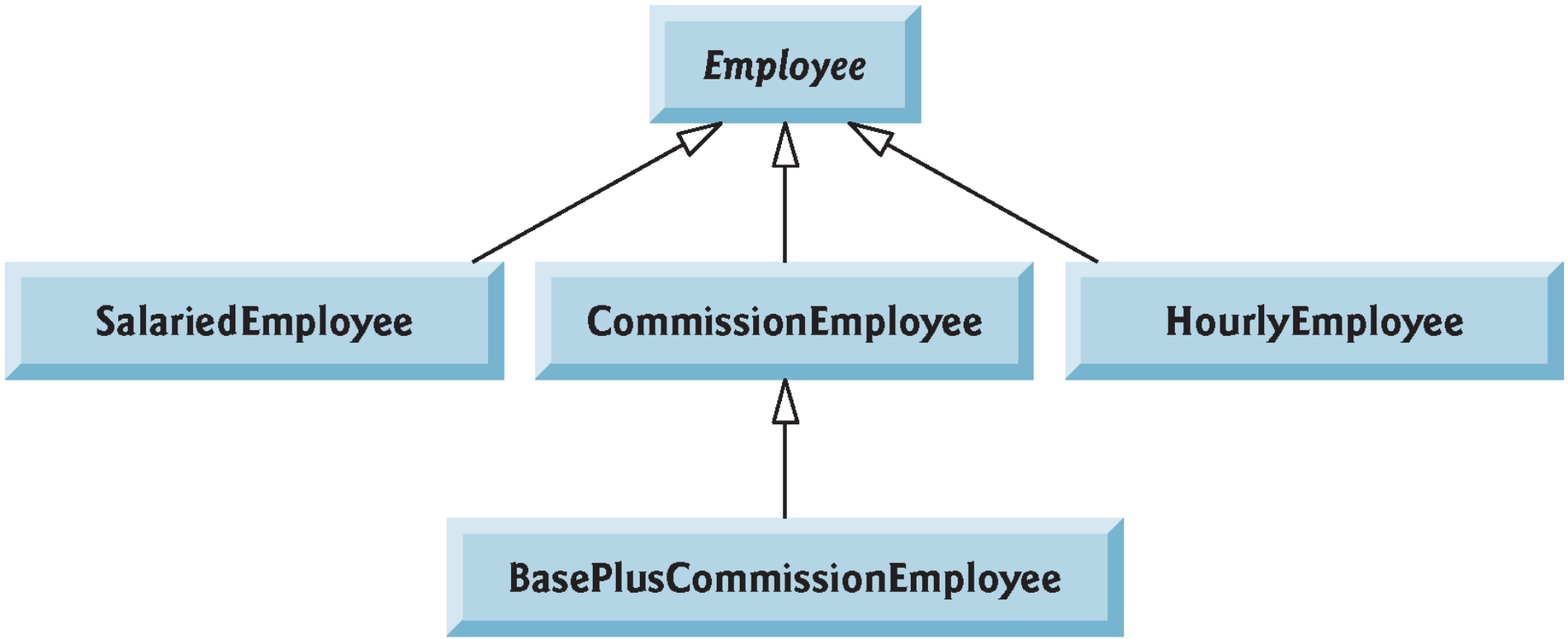


Fig. 10.2 | `Employee` hierarchy UML class diagram.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Case Study: Payroll System Using Polymorphism (Cont.)

- Abstract superclass `Employee` declares the "interface" to the hierarchy - that is, the set of methods that a program can invoke on all `Employee` objects.
  - We use the term "interface" here in a general sense to refer to the various ways programs can communicate with objects of any `Employee` subclass.
- Each employee has a first name, a last name and a social security number defined in abstract superclass `Employee`.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Abstract Superclass `Employee`

- The diagram in Fig. 10.3 shows each of the five `classes` in the hierarchy down the left side and methods `earnings` and `toString` across the top.
- For each class, the diagram shows the desired results of each method.
- Declaring the `earnings` method `abstract` indicates that each concrete subclass must provide an appropriate `earnings` implementation and that a program will be able to use superclass `Employee` variables to invoke method `earnings` polymorphically for any type of `Employee`.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

	earnings	toString
Employee	abstract	*firstName lastName* social security number: *SSN*
Salaried-Employee	weeklySalary	salaried employee: *firstName lastName* social security number: *SSN* weekly salary: *weeklySalary*

Hourly-Employee	if (hours <= 40) {
	  wage * hours
	}
	else if (hours > 40) {
	  40 * wage +
	  (hours - 40) *
	  wage * 1.5
	}
		hourly employee: *firstName lastName*
		social security number: *SSN*
		hourly wage: *wage*; hours worked: *hours*

Commission-Employee	commissionRate * grossSales	commission employee: *firstName lastName* social security number: *SSN* gross sales: *grossSales*; commission rate: *commissionRate*
BasePlus-Commission-Employee	(commissionRate * grossSales) + baseSalary	base salaried commission employee: *firstName lastName* social security number: *SSN* gross sales: *grossSales*; commission rate: *commissionRate*; base salary: *baseSalary*

Fig. 10.3 | Polymorphic interface for the `Employee` hierarchy classes.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Abstract Superclass `Employee`

- `Class Employee` (Fig. 10.4) provides methods `earnings` and `toString`, in addition to the *get* and *set* methods that manipulate `Employee`'s instance variables.
- An `earnings` method applies to all employees, but each earnings calculation depends on the employee's class.
  - An `abstract` method - there is no information to determine what amount `earnings` should return.
  - Each subclass overrides `earnings` with an appropriate implementation.
- Iterate through the array of `Employees` and call method `earnings` for each `Employee` subclass object.
  - Method calls processed polymorphically.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
1  // Fig. 10.4: Employee.java
2  // Employee abstract superclass.
3
4  public abstract class Employee {
5     private final String firstName;
6     private final String lastName;
7     private final String socialSecurityNumber;
8
9     // constructor
10    public Employee(String firstName, String lastName,
11       String socialSecurityNumber) {
12       this.firstName = firstName;
13       this.lastName = lastName;
14       this.socialSecurityNumber = socialSecurityNumber;
15    }
16
```

Fig. 10.4 | Employee abstract superclass. (Part 1 of 2.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
17      // return first name
18      public String getFirstName() {return firstName;}
19
20      // return last name
21      public String getLastName() {return lastName;}
22
23      // return social security number
24      public String getSocialSecurityNumber() {return socialSecurityNumber;}
25
26      // return String representation of Employee object
27      @Override
28      public String toString() {
29         return String.format("%s %s%nsocial security number: %s",
30            getFirstName(), getLastName(), getSocialSecurityNumber());
31      }
32
33      // abstract method must be overridden by concrete subclasses
34      public abstract double earnings(); // no implementation here
35   }
```

Fig. 10.4 | Employee abstract superclass. (Part 2 of 2.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
1  // Fig. 10.5: SalariedEmployee.java
2  // SalariedEmployee concrete class extends abstract class Employee.
3
4  public class SalariedEmployee extends Employee {
5     private double weeklySalary;
6
7     // constructor
8     public SalariedEmployee(String firstName, String lastName,
9        String socialSecurityNumber, double weeklySalary) {
10       super(firstName, lastName, socialSecurityNumber);
11
12       if (weeklySalary < 0.0) {
13          throw new IllegalArgumentException(
14             "Weekly salary must be >= 0.0");
15       }
16
17       this.weeklySalary = weeklySalary;
18    }
```

Fig. 10.5 | SalariedEmployee concrete class extends abstract class Employee. (Part 1 of 3.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
19
20      // set salary
21      public void setWeeklySalary(double weeklySalary) {
22         if (weeklySalary < 0.0) {
23            throw new IllegalArgumentException(
24               "Weekly salary must be >= 0.0");
25         }
26
27         this.weeklySalary = weeklySalary;
28      }
29
30      // return salary
31      public double getWeeklySalary() {return weeklySalary;}
32
```

Fig. 10.5 | SalariedEmployee concrete class extends abstract class Employee. (Part 2 of 3.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
33      // calculate earnings; override abstract method earnings in Employee
34      @Override
35      public double earnings() {return getWeeklySalary();}
36
37      // return String representation of SalariedEmployee object
38      @Override
39      public String toString() {
40         return String.format("salaried employee: %s%n%s: $%,.2f",
41            super.toString(), "weekly salary", getWeeklySalary());
42      }
43   }
```

Fig. 10.5 | SalariedEmployee concrete class extends abstract class Employee. (Part 3 of 3.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

10.5.3 Concrete Subclass HourlyEmployee

```
1  // Fig. 10.6: HourlyEmployee.java
2  // HourlyEmployee class extends Employee.
3
4  public class HourlyEmployee extends Employee {
5     private double wage; // wage per hour
6     private double hours; // hours worked for week
7
```

Fig. 10.6 | HourlyEmployee class extends Employee. (Part 1 of 5.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
 8      // constructor
 9      public HourlyEmployee(String firstName, String lastName,
10         String socialSecurityNumber, double wage, double hours) {
11         super(firstName, lastName, socialSecurityNumber);
12
13         if (wage < 0.0) { // validate wage
14            throw new IllegalArgumentException("Hourly wage must be >= 0.0");
15         }
16
17         if ((hours < 0.0) || (hours > 168.0)) { // validate hours
18            throw new IllegalArgumentException(
19               "Hours worked must be >= 0.0 and <= 168.0");
20         }
21
22         this.wage = wage;
23         this.hours = hours;
24      }
```

Fig. 10.6 | HourlyEmployee class extends Employee. (Part 2 of 5.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
25
26      // set wage
27      public void setWage(double wage) {
28         if (wage < 0.0) { // validate wage
29            throw new IllegalArgumentException("Hourly wage must be >= 0.0");
30         }
31
32         this.wage = wage;
33      }
34
35      // return wage
36      public double getWage() {return wage;}
37
```

Fig. 10.6 | HourlyEmployee class extends Employee. (Part 3 of 5.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
38      // set hours worked
39      public void setHours(double hours) {
40         if ((hours < 0.0) || (hours > 168.0)) { // validate hours
41            throw new IllegalArgumentException(
42               "Hours worked must be >= 0.0 and <= 168.0");
43         }
44
45         this.hours = hours;
46      }
47
48      // return hours worked
49      public double getHours() {return hours;}
50
```

Fig. 10.6 | HourlyEmployee class extends Employee. (Part 4 of 5.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
51      // calculate earnings; override abstract method earnings in Employee
52      @Override
53      public double earnings() {
54         if (getHours() <= 40) { // no overtime
55            return getWage() * getHours();
56         }
57         else {
58            return 40 * getWage() + (getHours() - 40) * getWage() * 1.5;
59         }
60      }
61
62      // return String representation of HourlyEmployee object
63      @Override
64      public String toString() {
65         return String.format("hourly employee: %s%n%s: $%,.2f; %s: %,.2f",
66            super.toString(), "hourly wage", getWage(),
67            "hours worked", getHours());
68      }
69   }
```

Fig. 10.6 | HourlyEmployee class extends Employee. (Part 5 of 5.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

10.5.4 Concrete Subclass `CommissionEmployee`

```
1  // Fig. 10.7: CommissionEmployee.java
2  // CommissionEmployee class extends Employee.
3
4  public class CommissionEmployee extends Employee {
5     private double grossSales; // gross weekly sales
6     private double commissionRate; // commission percentage
7
```

Fig. 10.7 | `CommissionEmployee` class extends `Employee`. (Part 1 of 5.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
 8      // constructor
 9      public CommissionEmployee(String firstName, String lastName,
10         String socialSecurityNumber, double grossSales,
11         double commissionRate) {
12         super(firstName, lastName, socialSecurityNumber);
13
14         if (commissionRate <= 0.0 || commissionRate >= 1.0) { // validate
15            throw new IllegalArgumentException(
16               "Commission rate must be > 0.0 and < 1.0");
17         }
18
19         if (grossSales < 0.0) { // validate
20            throw new IllegalArgumentException("Gross sales must be >= 0.0");
21         }
22
23         this.grossSales = grossSales;
24         this.commissionRate = commissionRate;
25      }
```

Fig. 10.7 | CommissionEmployee class extends Employee. (Part 2 of 5.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
26
27      // set gross sales amount
28      public void setGrossSales(double grossSales) {
29         if (grossSales < 0.0) { // validate
30            throw new IllegalArgumentException("Gross sales must be >= 0.0");
31         }
32
33         this.grossSales = grossSales;
34      }
35
36      // return gross sales amount
37      public double getGrossSales() {return grossSales;}
38
```

Fig. 10.7 | CommissionEmployee class extends Employee. (Part 3 of 5.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
39      // set commission rate
40      public void setCommissionRate(double commissionRate) {
41         if (commissionRate <= 0.0 || commissionRate >= 1.0) { // validate
42            throw new IllegalArgumentException(
43               "Commission rate must be > 0.0 and < 1.0");
44         }
45
46         this.commissionRate = commissionRate;
47      }
48
49      // return commission rate
50      public double getCommissionRate() {return commissionRate;}
51
```

Fig. 10.7 | CommissionEmployee class extends Employee. (Part 4 of 5.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
52      // calculate earnings; override abstract method earnings in Employee
53      @Override
54      public double earnings() {
55         return getCommissionRate() * getGrossSales();
56      }
57
58      // return String representation of CommissionEmployee object
59      @Override
60      public String toString() {
61         return String.format("%s: %s%n%s: $%,.2f; %s: %.2f",
62            "commission employee", super.toString(),
63            "gross sales", getGrossSales(),
64            "commission rate", getCommissionRate());
65      }
66   }
```

Fig. 10.7 | CommissionEmployee class extends Employee. (Part 5 of 5.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

10.5.5 Indirect Concrete Subclass BasePlusCommissionEmployee

```
1  // Fig. 10.8: BasePlusCommissionEmployee.java
2  // BasePlusCommissionEmployee class extends CommissionEmployee.
3
4  public class BasePlusCommissionEmployee extends CommissionEmployee {
5     private double baseSalary; // base salary per week
6
7     // constructor
8     public BasePlusCommissionEmployee(String firstName, String lastName,
9        String socialSecurityNumber, double grossSales,
10       double commissionRate, double baseSalary) {
11       super(firstName, lastName, socialSecurityNumber,
12          grossSales, commissionRate);
13
14       if (baseSalary < 0.0) { // validate baseSalary
15          throw new IllegalArgumentException("Base salary must be >= 0.0");
16       }
17
18       this.baseSalary = baseSalary;
19    }
```

```
20
21      // set base salary
22      public void setBaseSalary(double baseSalary) {
23         if (baseSalary < 0.0) { // validate baseSalary
24            throw new IllegalArgumentException("Base salary must be >= 0.0");
25         }
26
27         this.baseSalary = baseSalary;
28      }
29
30      // return base salary
31      public double getBaseSalary() {return baseSalary;}
32
```

Fig. 10.8 | BasePlusCommissionEmployee class extends CommissionEmployee. (Part 2 of 3.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
33      // calculate earnings; override method earnings in CommissionEmployee
34      @Override
35      public double earnings() {return getBaseSalary() + super.earnings();}
36
37      // return String representation of BasePlusCommissionEmployee object
38      @Override
39      public String toString() {
40         return String.format("%s %s; %s: $%,.2f",
41            "base-salaried", super.toString(),
42            "base salary", getBaseSalary());
43      }
44   }
```

Fig. 10.8 | BasePlusCommissionEmployee class extends CommissionEmployee. (Part 3 of 3.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

10.5.6 Polymorphic Processing, Operator `instanceof` and Downcasting

- Fig. 10.9 creates an object of each of the four concrete classes, manipulates these objects *nonpolymorphically*, via variables of each object's own type, then *polymorphically*, using an array of `Employee` variables.
- While processing the objects polymorphically, the program increases the base salary of each `BasePlusCommissionEmployee` by 10%
  - Requires *determining the object's type at execution time*.
- Finally, the program polymorphically determines and outputs the *type* of each object in the `Employee` array.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
1  // Fig. 10.9: PayrollSystemTest.java
2  // Employee hierarchy test program.
3
4  public class PayrollSystemTest {
5     public static void main(String[] args) {
6        // create subclass objects
7        SalariedEmployee salariedEmployee =
8           new SalariedEmployee("John", "Smith", "111-11-1111", 800.00);
9        HourlyEmployee hourlyEmployee =
10          new HourlyEmployee("Karen", "Price", "222-22-2222", 16.75, 40);
11       CommissionEmployee commissionEmployee =
12          new CommissionEmployee(
13          "Sue", "Jones", "333-33-3333", 10000, .06);
14       BasePlusCommissionEmployee basePlusCommissionEmployee =
15          new BasePlusCommissionEmployee(
16          "Bob", "Lewis", "444-44-4444", 5000, .04, 300);
17
```

Fig. 10.9 | Employee hierarchy test program. (Part 1 of 8.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
18          System.out.println("Employees processed individually:");
19
20          System.out.printf("%n%s%n%s: $%,.2f%n%n",
21             salariedEmployee, "earned", salariedEmployee.earnings());
22          System.out.printf("%s%n%s: $%,.2f%n%n",
23             hourlyEmployee, "earned", hourlyEmployee.earnings());
24          System.out.printf("%s%n%s: $%,.2f%n%n",
25             commissionEmployee, "earned", commissionEmployee.earnings());
26          System.out.printf("%s%n%s: $%,.2f%n%n",
27             basePlusCommissionEmployee,
28             "earned", basePlusCommissionEmployee.earnings());
29
```

Fig. 10.9 | Employee hierarchy test program. (Part 2 of 8.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
30          // create four-element Employee array
31          Employee[] employees = new Employee[4];
32
33          // initialize array with Employees
34          employees[0] = salariedEmployee;
35          employees[1] = hourlyEmployee;
36          employees[2] = commissionEmployee;
37          employees[3] = basePlusCommissionEmployee;
38
39          System.out.printf("Employees processed polymorphically:%n%n");
40
```

Fig. 10.9 | Employee hierarchy test program. (Part 3 of 8.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
41          // generically process each element in array employees
42          for (Employee currentEmployee : employees) {
43             System.out.println(currentEmployee); // invokes toString
44
45             // determine whether element is a BasePlusCommissionEmployee
46             if (currentEmployee instanceof BasePlusCommissionEmployee) {
47                // downcast Employee reference to
48                // BasePlusCommissionEmployee reference
49                BasePlusCommissionEmployee employee =
50                   (BasePlusCommissionEmployee) currentEmployee;
51
52                employee.setBaseSalary(1.10 * employee.getBaseSalary());
53
54                System.out.printf(
55                   "new base salary with 10%% increase is: $%,.2f%n",
56                   employee.getBaseSalary());
57             }
58
59             System.out.printf(
60                "earned $%,.2f%n%n", currentEmployee.earnings());
61          }
```

Fig. 10.9 | Employee hierarchy test program. (Part 4 of 8.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Why do we need `downcasting`?

```
62
63          // get type name of each object in employees array
64          for (int j = 0; j < employees.length; j++) {
65             System.out.printf("Employee %d is a %s%n", j,
66                employees[j].getClass().getName());
67          }
68       }
69    }
```

Fig. 10.9 | Employee hierarchy test program. (Part 5 of 8.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
Employees processed individually:

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: $800.00
earned: $800.00

hourly employee: Karen Price
social security number: 222-22-2222
hourly wage: $16.75; hours worked: 40.00
earned: $670.00

commission employee: Sue Jones
social security number: 333-33-3333
gross sales: $10,000.00; commission rate: 0.06
earned: $600.00

base-salaried commission employee: Bob Lewis
social security number: 444-44-4444
gross sales: $5,000.00; commission rate: 0.04; base salary: $300.00
earned: $500.00
```

Fig. 10.9 | Employee hierarchy test program. (Part 6 of 8.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
Employees processed polymorphically:

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: $800.00
earned $800.00

hourly employee: Karen Price
social security number: 222-22-2222
hourly wage: $16.75; hours worked: 40.00
earned $670.00

commission employee: Sue Jones
social security number: 333-33-3333
gross sales: $10,000.00; commission rate: 0.06
earned $600.00

base-salaried commission employee: Bob Lewis
social security number: 444-44-4444
gross sales: $5,000.00; commission rate: 0.04; base salary: $300.00
new base salary with 10% increase is: $330.00
earned $530.00
```

Fig. 10.9 | Employee hierarchy test program. (Part 7 of 8.)

```
Employee 0 is a SalariedEmployee
Employee 1 is a HourlyEmployee
Employee 2 is a CommissionEmployee
Employee 3 is a BasePlusCommissionEmployee
```

Fig. 10.9 | `Employee` hierarchy test program. (Part 8 of 8.)

Polymorphic Processing, Operator `instanceof` and Downcasting (Cont.)

- Every object *knows its own class* and can access this information through the `getClass` method, which all classes inherit from class `Object`.
  - The `getClass` method returns an object of type `Class` (from package `java.lang`), which contains information about the object's type, including its class name.
  - The result of the `getClass` call is used to invoke `getName` to get the object's class name.

`final` Methods and Classes

- A `final` method in a superclass cannot be overridden in a subclass.
  - Methods that are declared `private` are implicitly `final`, because it's not possible to override them in a subclass.
  - Methods that are declared `static` are implicitly `final`.
  - A `final` method's declaration can never change, so all subclasses use the same method implementation, and calls to `final` methods are resolved at compile time, this is known as static binding.

Example

```
class Employee {
    public final void display() {
        System.out.println("Employee");
    }
}

class Manager extends Employee {
    //  Cannot override display()
}
```

A final method means: You cannot change/override this method in a subclass.

Example

```
class Employee {
private void calculate() { }
 }
```

The subclass doesn't see this method, so it cannot override it.

```
class Employee {
    static void show() {
        System.out.println("Employee");
    }
}

class Manager extends Employee {
    static void show() {
        System.out.println("Manager");
    }
}
```

This is method hiding, not overriding.

`final` Methods and Classes (Cont.)

- A `final` class cannot be extended to create a subclass.
  - All methods in a `final` class are implicitly `final`.
- Class `String` is an example of a `final` class.
  - If you were allowed to create a subclass of `String`, objects of that subclass could be used wherever `Strings` are expected.
  - Since class `String` cannot be extended, programs that use `Strings` can rely on the functionality of `String` objects as specified in the Java API.
  - Making the class `final` also prevents programmers from creating subclasses that might bypass security restrictions.

Attempting to declare a subclass of a `final` class is a compilation error.

Example

```
final class String {
    // ...
}
class MyString extends String {   //  Not allowed
}
```

Nobody can modify String's behavior through inheritance.

h



Software Engineering Observation 10.6

In the Java API, the vast majority of classes are not declared `final`. This enables inheritance and polymorphism. However, in some cases, it's important to declare classes `final`—typically for security reasons. Also, unless you carefully design a class for extension, you should declare the class as `final` to avoid (often subtle) errors.

Creating and Using Interfaces

- Our next example reexamines the previous payroll system.
- Suppose that the company wishes to perform several accounting operations in a single ***accounts payable*** (money yet to pay) application
  - Calculating the earnings that must be paid to each employee
  - Calculate the payment due on each of several invoices (i.e., bills for goods purchased)
- Both operations have to do with obtaining some kind of payment amount.
  - For an employee, the payment refers to the employee's earnings.
  - For an invoice, the payment refers to the total cost of the goods listed on the invoice.

Exercise ReThink about Figure 10.2, Redesign UML Diagram

Employee and Invoice are completely unrelated classes : one is a person, one is a bill ; yet an accounts-payable program needs to calculate a payment amount for both. We don't want Invoice to inherit from Employee, since an invoice clearly isn't a kind of employee.

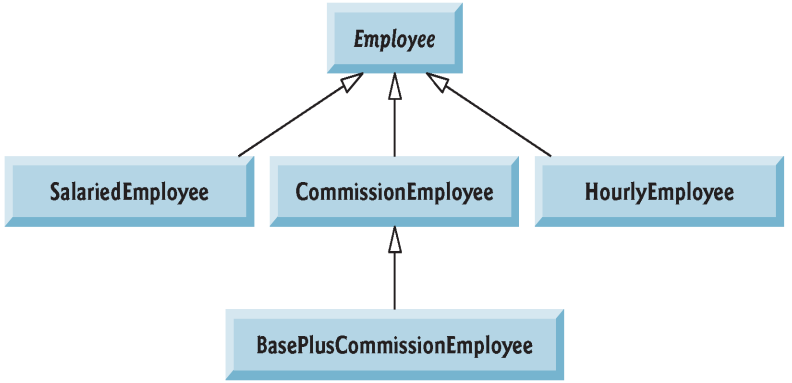


Fig. 10.2 | Employee hierarchy UML class diagram.

What could let two unrelated classes share a common capability, without one inheriting from the other?

Developing a Payable Hierarchy

- The next example builds an application that can determine payments for both employees and invoices.
  - Classes `Invoice` and `Employee` both represent things for which the company must be able to calculate a payment amount.
  - Both classes implement the `Payable` interface, so a program can invoke method `getPaymentAmount` on `Invoice` objects and `Employee` objects alike.
  - Enables the polymorphic processing of `Invoices` and `Employees`.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Developing a Payable Hierarchy (Cont.)

- Fig. 10.10 shows the ***accounts payable*** hierarchy.
- UML distinguishes an interface from other classes by placing «interface» above the interface name.
- The UML expresses the relationship between a class and an interface through a realization.
  - A class is said to “realize” (or implement in Java) the methods of an interface.
  - A class diagram models a realization as a dashed arrow with a hollow arrowhead pointing from the implementing class to the interface.
- A subclass inherits its superclass’s realization relationships.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

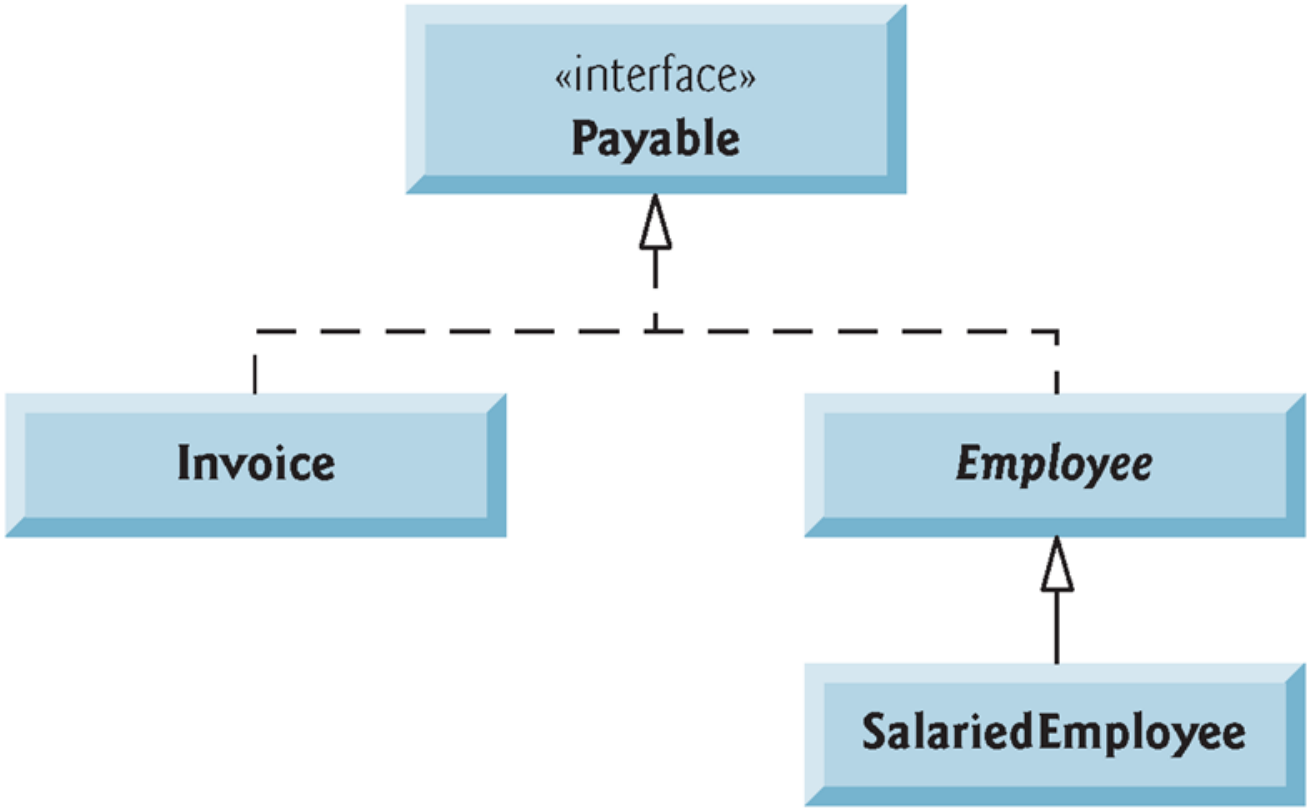


Fig. 10.10 | Payable hierarchy UML class diagram.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Note:

1. in a Java **interface**, every method you declare without modifiers is **implicitly `public` and `abstract`**.
2. in an interface, **every field is implicitly public, static, and final**, even if you don't write those keywords.



Good Programming Practice 10.2

Use `public` and `abstract` explicitly when declaring interface methods to make your intentions clear. As you'll see in Sections 10.10–10.11, Java SE 8 and Java SE 9 allow other kinds of methods in interfaces.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Interface Payable (or IPayable)

- Interfaces can have any number of methods (public and private since Java 8). They can also have static methods since Java 8.
- Interfaces may also contain `final` and `static` constants

```
public interface Payable {

    public int x = 0; // Will be a final static field because this is an interface
    public Double getPaymentAmount();

}
```



```
public interface IPayable {

    public static final int x = 0;
    public abstract Double getPaymentAmount();

}
```

y Pearson Education, Inc. All Rights Reserved.

Class Invoice

- Class `Invoice` (Fig. 10.12) represents a simple invoice that contains billing information for only one kind of part (invoice item)
- Java does not allow subclasses to inherit from more than one superclass, but it allows a class to inherit from one superclass and implement as many interfaces as it needs.
- To implement more than one, use a comma-separated list of interface names after keyword `implements` in the class declaration, as in:

```
public class ClassName extends SuperclassName
   implements FirstInterface, SecondInterface, ...
```

```
1  // Fig. 10.12: Invoice.java
2  // Invoice class that implements Payable.
3
4  public class Invoice implements Payable {
5     private final String partNumber;
6     private final String partDescription;
7     private final int quantity;
8     private final double pricePerItem;
9
```

Fig. 10.12 | Invoice class that implements Payable. (Part 1 of 4.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
10      // constructor
11      public Invoice(String partNumber, String partDescription, int quantity,
12         double pricePerItem) {
13         if (quantity < 0) { // validate quantity
14            throw new IllegalArgumentException("Quantity must be >= 0");
15         }
16
17         if (pricePerItem < 0.0) { // validate pricePerItem
18            throw new IllegalArgumentException(
19               "Price per item must be >= 0");
20         }
21
22         this.quantity = quantity;
23         this.partNumber = partNumber;
24         this.partDescription = partDescription;
25         this.pricePerItem = pricePerItem;
26      }
```

Fig. 10.12 | Invoice class that implements Payable. (Part 2 of 4.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
27
28      // get part number
29      public String getPartNumber() {return partNumber;}
30
31      // get description
32      public String getPartDescription() {return partDescription;}
33
34      // get quantity
35      public int getQuantity() {return quantity;}
36
37      // get price per item
38      public double getPricePerItem() {return pricePerItem;}
39
```

Fig. 10.12 | Invoice class that implements Payable. (Part 3 of 4.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
40      // return String representation of Invoice object
41      @Override
42      public String toString() {
43         return String.format("%s: %n%s: %s (%s) %n%s: %d %n%s: $%,.2f",
44            "invoice", "part number", getPartNumber(), getPartDescription(),
45            "quantity", getQuantity(), "price per item", getPricePerItem());
46      }
47
48      // method required to carry out contract with interface Payable
49      @Override
50      public double getPaymentAmount() {
51         return getQuantity() * getPricePerItem(); // calculate total cost
52      }
53   }
```

Fig. 10.12 | `Invoice` class that implements `Payable`. (Part 4 of 4.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Modifying Class Employee to Implement Interface Payable

- We now modify class Employee to implement interface Payable. This class declaration is the same with two changes:
  - Line 4 of indicates that class Employee now implements Payable.
  - Line 38 implements interface Payable's getPaymentAmount method.
- getPaymentAmount simply calls Employee's abstract method earnings
  - At execution time, when getPaymentAmount is called on an object of an Employee subclass, getPaymentAmount calls that subclass's concrete earnings method, which knows how to calculate earnings for objects of that subclass type.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
1  // Fig. 10.13: Employee.java
2  // Employee abstract superclass that implements Payable.
3
4  public abstract class Employee implements Payable {
5     private final String firstName;
6     private final String lastName;
7     private final String socialSecurityNumber;
8
9     // constructor
10    public Employee(String firstName, String lastName,
11       String socialSecurityNumber) {
12       this.firstName = firstName;
13       this.lastName = lastName;
14       this.socialSecurityNumber = socialSecurityNumber;
15    }
16
```

Fig. 10.13 | Employee abstract superclass that implements Payable. (Part 1 of 3.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
17      // return first name
18      public String getFirstName() {return firstName;}
19
20      // return last name
21      public String getLastName() {return lastName;}
22
23      // return social security number
24      public String getSocialSecurityNumber() {return socialSecurityNumber;}
25
26      // return String representation of Employee object
27      @Override
28      public String toString() {
29         return String.format("%s %s%nsocial security number: %s",
30            getFirstName(), getLastName(), getSocialSecurityNumber());
31      }
32
```

Fig. 10.13 | `Employee` abstract superclass that implements `Payable`. (Part 2 of 3.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
33      // abstract method must be overridden by concrete subclasses
34      public abstract double earnings(); // no implementation here
35
36      // implementing getPaymentAmount here enables the entire Employee
37      // class hierarchy to be used in an app that processes Payables
38      public double getPaymentAmount() {return earnings();}
39  }
```

Fig. 10.13 | `Employee` `abstract` superclass that implements `Payable`. (Part 3 of 3.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.



Software Engineering Observation 10.12

Using a superclass reference, we can polymorphically invoke any method declared in the superclass and its superclasses (e.g., class `Object`). Using an interface reference, we can polymorphically invoke any method declared in the interface, its superinterfaces (one interface can extend another) and in class `Object`—a variable of an interface type must refer to an object to call methods, and all objects have the methods of class `Object`.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

- **Rule:** With a superclass reference, you can only call methods that the superclass declares (or inherits from Object).

- **Rule:** An interface reference can invoke only methods declared in the interface (or its superinterfaces).

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Using Interface Payable to Process Invoices and Employees Polymorphically

- PayableInterfaceTest (Fig. 10.14) illustrates that interface Payable can be used to process a set of Invoices and Employees *polymorphically* in a single application.
- Lines 18–23 *polymorphically* process each Payable object in payableObjects, displaying each object's String representation and payment amount
- Line 21 invokes method toString via a Payable interface reference, even though toString is not declared in interface Payable—*all references (including those of interface types) refer to objects that extend Object and therefore have a toString method*

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Using Interface Payable to Process Invoices and Employees Polymorphically

- Line 22 invokes Payable method getPaymentAmount to obtain the payment amount for each object in payableObjects, regardless of the actual type of the object. The output reveals that each of the method calls in lines 21–22 invokes the appropriate class's toString and getPayment-Amount methods.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
1  // Fig. 10.14: PayableInterfaceTest.java
2  // Payable interface test program processing Invoices and
3  // Employees polymorphically.
4  public class PayableInterfaceTest {
5     public static void main(String[] args) {
6        // create four-element Payable array
7        Payable[] payableObjects = new Payable[] {
8           new Invoice("01234", "seat", 2, 375.00),
9           new Invoice("56789", "tire", 4, 79.95),
10          new SalariedEmployee("John", "Smith", "111-11-1111", 800.00),
11          new SalariedEmployee("Lisa", "Barnes", "888-88-8888", 1200.00)
12       };
13
```

Fig. 10.14 | `Payable` interface test program processing `Invoices` and `Employees` polymorphically. (Part 1 of 3.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
14          System.out.println(
15             "Invoices and Employees processed polymorphically:");
16
17          // generically process each element in array payableObjects
18          for (Payable currentPayable : payableObjects) {
19             // output currentPayable and its appropriate payment amount
20             System.out.printf("%n%s %npayment due: $%,.2f%n",
21                currentPayable.toString(), // could invoke implicitly
22                currentPayable.getPaymentAmount());
23          }
24       }
25    }
```

Fig. 10.14 | `Payable` interface test program processing `Invoices` and `Employees` polymorphically. (Part 2 of 3.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

```
Invoices and Employees processed polymorphically:

invoice:
part number: 01234 (seat)
quantity: 2
price per item: $375.00
payment due: $750.00

invoice:
part number: 56789 (tire)
quantity: 4
price per item: $79.95
payment due: $319.80

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: $800.00
payment due: $800.00

salaried employee: Lisa Barnes
social security number: 888-88-8888
weekly salary: $1,200.00
payment due: $1,200.00
```

Fig. 10.14 | `Payable` interface test program processing `Invoices` and `Employees` polymorphically. (Part 3 of 3.)

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

10.9.7 Some Common Interfaces of the Java API

- You'll use interfaces extensively when developing Java applications. The Java API contains numerous interfaces, and many of the Java API methods take interface arguments and return interface values.
- Figure 10.16 overviews a few of the most popular interfaces of the Java API that we use in later chapters.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Interface	Description
Comparable	Java contains several comparison operators (e.g., <, <=, >, >=, ==, !=) that allow you to compare primitive values. However, these operators cannot be used to compare objects. The interface Comparable is used to allow objects of a class that implements the interface to be compared to one another. It is commonly used for ordering objects in sorting algorithms.
Serializable	An interface used to identify classes whose objects can be written to (i.e., serialized) or read from (i.e., deserialized) some type of storage (e.g., file on disk, database field) or transmitted across a network.
Runnable	Implemented by any class that represents a task to perform. Objects of such a class are executed in parallel using multithreading (chapter 5: concurrency). The interface contains one method, “run”, which specifies the behavior of an object when executed.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

`Default` Interface Methods

- Prior to Java SE 8, interface methods could be *only* **`public abstract`** methods.
  - An interface specified *what* operations an implementing class must perform but not *how* the class should perform them.
- Since Java SE 8, interfaces also may contain `public default` methods with concrete default implementations that specify how operations are performed when an implementing class does not override the methods.
- If a class implements such an interface, the class also receives the interface's `default` implementations (if any).
- To declare a default method, place the `keyword default` before the method's return type and provide a concrete method implementation.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

`Default` Interface Methods (Cont.)

Adding Methods to Existing Interfaces

- Any class that implements the original interface will *not* break when a `default` method is added.
  - The class simply receives the new default method.
- When a class implements a Java SE 8 interface, the class "signs a contract" with the compiler that says,
  - "I will declare all the *`abstract`* methods specified by the interface or I will declare my class `abstract`"
- The implementing class is not required to override the interface's `default` methods, but it can if necessary.

Java SE 8 **`default`** methods enable you to evolve existing interfaces by adding new methods to those interfaces without breaking code that uses them.

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

`Default` Interface Methods (Cont.)

Interfaces vs. `abstract` Classes

- Prior to Java SE 8, an interface was typically used (rather than an `abstract` class) when there were no implementation details to inherit—no fields and no method implementations.
- With `default` methods, you can instead declare common method implementations in interfaces
- This gives you more flexibility in designing your classes, because a class can implement many interfaces, but can extend only one superclass

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

`Static` Interface Methods (Cont.)

- Prior to Java SE 8, it was common to associate with an interface a class containing `static` helper methods for working with objects that implemented the interface.
- The class `Collections` contains many `static` helper methods for working with objects that implement interfaces `Collection`, `List`, `Set` and more.
- `Collections` method **`sort`** can sort objects of *any* class that implements interface `List`.
- With `static` interface methods, such helper methods can now be declared directly in interfaces rather than in separate classes.

```
interface MathUtil {
    static int square(int x) {
        return x * x;
    }
}
 int result = MathUtil.square(5);
```

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Example

```
interface Shape {
    double area();

    static void printArea(Shape s) {
        System.out.println(s.area());
    }
}

class Circle implements Shape {
    public double area() {
        return 3.14 * 5 * 5;
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        Circle c = new Circle();

        Shape.printArea(c);
    }
```

Functional Interfaces

- As of Java SE 8, any interface containing only one `abstract` method is known as a functional interface, also called SAM (Single Abstract Method) interfaces
- Functional interfaces include:
  - `ActionListener` — used to define a method that's called when the user clicks a button.
  - `Comparator` — used to define a method that can compare two objects of a given type to determine whether the first object is less than, equal to or greater than the second.
  - `Runnable` — used to define a task that may be run in parallel with other parts of your program.

```
public interface ActionListener extends EventListener {

    /**
     * Invoked when an action occurs.
     * @param e the event to be processed
     */
    public void actionPerformed(ActionEvent e);

}
```

Java SE 9 private Interface Methods

- A class's `private` helper methods may be called only by the class's other methods
- As of Java SE 9, you can declare helper methods in *interfaces* via **`private` interface methods**
- An interface's `private` instance methods can be called directly (i.e., without an object reference) only by the interface's other instance methods
- An interface's `private static` methods can be called by any of the interface's instance or `static` methods

> Including the `default` keyword in a `private` interface method's declaration is a compilation error—`default` methods must be `public`.

Java SE 9 private Interface Methods Example

```
interface MyInterface {

    private void helper() {
        System.out.println("Helper");
    }

    default void method1() {
        helper();       //  OK
    }

    static void method2() {
        helper();       //  Compile error
    }
}
```

```
interface MyInterface {

    private static void helper() {
        System.out.println("Helper");
    }

    default void method1() {
        helper();       // OK
    }

    static void method2() {
        helper();       // OK
    }
}
```

Private Constructors

Sometimes it's useful to declare one or more of a class's constructors as private.

Preventing Object Instantiation

- You can prevent client code from creating objects of a class by making the class's constructors private
- Consider class `Math`, which contains only `public static` constants and `public static` methods
- There's no need to create a `Math` object to use the class's constants and methods, so its constructor is `private`

Example

Sometimes a class is designed only to provide: static methods and static constants and does not represent an object.

```
class Calculator {
    private Calculator() {
        // Prevent object creation
    }
    public static int add(int a, int b) {
        return a + b;
    }
}

//Calculator c = new Calculator();//ERROR
int result = Calculator.add(5, 3);
 System.out.println(result);
```

Exercise (Private Constructors) [6 min]

- Create a class called UtilityLogger that can be used to log messages using two static methods *logInfo and logError* that log message into the console.
- Prevent instantiation of the class

    new UtilityLogger(); // should give a compilation error

Solution

```
class UtilityLogger {

    // Private constructor
    private UtilityLogger() {
    }

    // Static method
    public static void logInfo(String message) {
        System.out.println("INFO: " + message);
    }

    // Static method
    public static void logError(String message) {
        System.out.println("ERROR: " + message);
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        UtilityLogger.logInfo("Program started");

        UtilityLogger.logError("File not found");
    }
}
```

Private Constructors (cont.)

Sharing Initialization Code in Constructors

- One common use of a `private` constructor is sharing initialization code among a class's other constructors
- You can use delegating constructors to call the `private` constructor that contains the shared initialization code

Private Constructors Example

```
class Student {
    private String name;
    private int age;
    // Private constructor containing common initialization
    private Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
public Student(String name) {
        this(name, 18);   // delegates to private constructor
    }
}
```

Exercise (constructors sharing initialization code) [5 min]

- Create a class called UserProfile that can be used to create users.
- Add a constructor that takes a username, name and email and prints “Initializing user profile: *username* (*name*), email: *email@domain.sa*”
- Add a second constructor that takes only a username and email and uses a default name (same as username).
- **Requirement:** The two constructors should share the same initialization code using **constructor chaining** (this(...)). Avoid duplicating the initialization/printing code in both constructors.

Solution

```
public class UserProfile {
    private String username;
    private String name;
    private String email;
    // Private constructor containing shared initialization code
    private UserProfile(String username, String name,
                                String email, boolean init) {
        this.username = username;
        this.name = name;
        this.email = email;
        System.out.println(           "Initializing user profile: " +
            username + " (" + name +
            "), email: " + email
        );
    }

    // Constructor 1: username, name, email
    public UserProfile(String username, String name, String email) {

        this(username, name, email, true);
    }

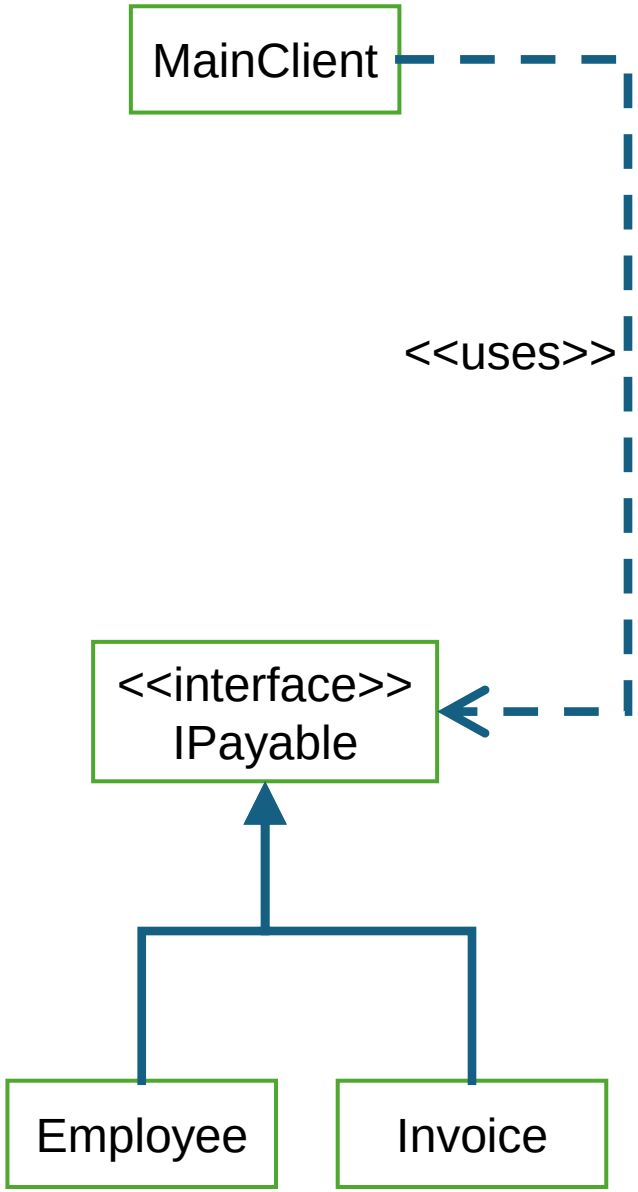
    // Constructor 2: username, email
    public UserProfile(String username, String email) {

        this(username, username, email, true);
    }
}
```

Private Constructors (cont.)

Factory Methods

- Another common use of `private` constructors is to force client code to use so-called “factory methods” to create objects
- A **factory method** is a public static method that creates and initializes an object of a specified type (possibly of the same class), then returns a reference to it
- A key benefit of this architecture is that the method’s return type can be an interface or a superclass (either abstract or concrete) -> client code does not need to be hard-wired to the class type but can depend on the interface abstraction.



Simple Example

```
interface Shape {
    void draw();
}
class Circle implements Shape {

    public void draw() {
        System.out.println("Drawing Circle");
    }
}
class Rectangle implements Shape {

    public void draw() {
        System.out.println("Drawing Rectangle");
    }
}
```

```
class ShapeFactory {
//The return type is Shape, not Circle or Rectangle.
    public static Shape createShape(String type) {
        if (type.equals("circle"))
            return new Circle();
        if (type.equals("rectangle"))
            return new Rectangle();
        return null;
    }
}
public class Main {
    public static void main(String[] args) {
 //The client only knows about shape
        Shape s1 = ShapeFactory.createShape("circle");
        Shape s2 = ShapeFactory.createShape("rectangle");
        s1.draw();
        s2.draw();
    }
}
```

Exercise (***Factory Methods***) [6 min]

- Create an abstract Logger class with a static factory method that can return a concrete ConsoleLogger or a concrete FileLogger depending on a given argument (type).

Solution: project ch04/factory

Solution

```
public abstract class Logger {
    public abstract void log(String message);
    // Static factory method
    public static Logger getLogger(String type) {
        if (type == null) {
            throw new IllegalArgumentException("Logger type cannot be null");
        }
        switch (type.toLowerCase()) {
            case "console":
                return new ConsoleLogger();
            case "file":
                return new FileLogger("app.log");
            default:
                throw new IllegalArgumentException("Unknown logger type: " + type);
        }
    }
}
```

Solution (2)

```
class ConsoleLogger extends Logger {

    @Override

    public void log(String message) {

        System.out.println("[Console] " + message);

    }

}
```

```
class FileLogger extends Logger {
    private final String filename;

    public FileLogger(String filename) {
        this.filename = filename;
    }
    @Override
    public void log(String message) {
        try (java.io.FileWriter fw = new java.io.FileWriter(filename, true)) {
            fw.write("[File] " + message + System.lineSeparator());
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

Program to an Interface, Not an Implementation (not an abstract class)

- Recall that Java does not allow a class to inherit from more than one superclass
- With interface implementation, a class implements an interface describing various abstract methods that the new class must provide
- The new class also may inherit some method implementations (allowed in interfaces as of Java SE 8), but no instance variables (interface properties are considered static).
- Recall that Java allows a class to implement multiple interfaces in addition to extending one class
- An interface also may extend one or more other interfaces.

Implementation Inheritance Is Best for Small Numbers of Tightly Coupled Classes

- Implementation inheritance is primarily used to declare closely related classes (example of SalariedEmployee, CommissionEmployee…)
  - They share many of the same instance variables and method implementations
- Every subclass object has the ***is-a*** relationship with the superclass
  - anywhere a superclass object is expected, a subclass object may be provided
- Changes to a superclass directly affect all corresponding subclasses
- When you use a superclass variable, only a superclass object or one of its subclass objects may be assigned to the variable.

Implementation Inheritance Is Best for Small Numbers of Tightly Coupled Classes (cont.)

- A key disadvantage of implementation inheritance is that the tight coupling among the classes can make it difficult to modify the hierarchy
- Small inheritance hierarchies under the control of one person tend to be more manageable than large ones maintained by many people



Software Engineering Observation 10.14

Java SE 8's and Java SE 9's `interface` enhancements (Sections 10.10–10.11)—which allow `interfaces` to contain `public` and `private` instance methods and `static` methods with implementations—make programming with `interfaces` appropriate for almost all cases in which you would have used `abstract` classes previously. With the exception of fields, you get all the benefits that classes provide, plus classes can implement any number of interfaces but can extend only one class (`abstract` or concrete).

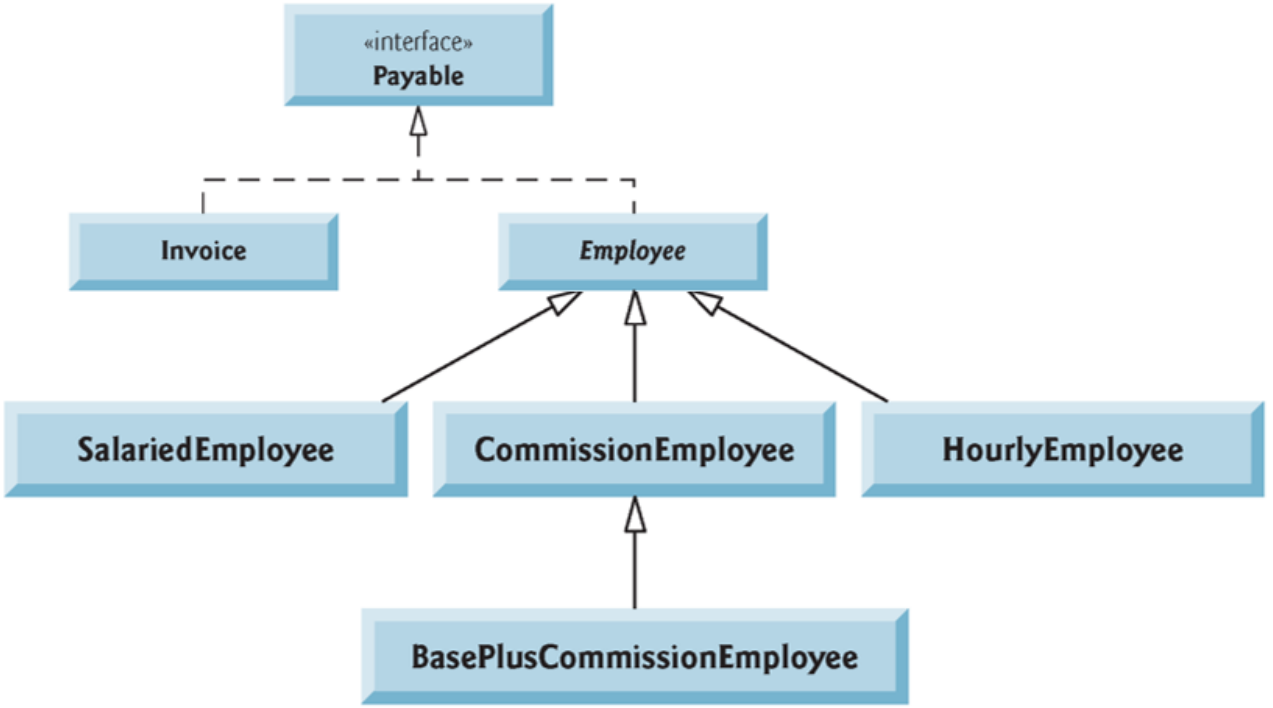


Software Engineering Observation 10.15

Just as superclasses can change, so can interfaces. If the signature of an interface method changes, all corresponding classes would require modification. Experience has shown that interfaces change much less frequently than implementations.

Rethinking the Employee Hierarchy

- Let's reconsider the `Employee` hierarchy with composition and an interface: *In reality, the employees are not different, it is their compensation model that is different.*
- Each type of employee is an `Employee` that ***has a*** `CompensationModel`
- We can declare `CompensationModel` as an interface with an `abstract` **`earnings`** method, then declare implementations of `CompensationModel` that specify the various ways in which an Employee gets paid



Rethinking the Employee Hierarchy (2)

- A `SalariedCompensationModel` would contain a `weeklySalary` instance variable and would implement `earnings` to return the `weeklySalary`.
- An `HourlyCompensationModel` would contain `wage` and `hours` instance variables and would implement `earnings` based on the number of hours worked, with `1.5 * wage` for any hours over 40.
- A `CommissionCompensationModel` would contain `grossSales` and `commissionRate` instance variables and would implement `earnings` to return `grossSales * commissionRate`.
- A `BasePlusCommissionCompensationModel` would contain instance variables `grossSales`, `commissionRate` and `baseSalary` and would implement `earnings` to return `baseSalary + grossSales * commissionRate`

Rethinking the Employee Hierarchy (3)

- Each `Employee` object you create can then be initialized with an object of the appropriate `CompensationModel` implementation
- Class `Employee`'s `earnings` method would simply use the class's composed `CompensationModel` instance variable to call the `earnings` method of the corresponding `CompensationModel` object

Rethinking the Employee Hierarchy (4)

Flexibility if Compensation Models Change

- Declaring the `CompensationModels` as separate classes that implement the same interface provides flexibility for future changes
- Assume `Employees` who are paid by commission based on gross sales should get an extra 10% commission, but those who have a base salary should not
- In the original `Employee` hierarchy, making this change to class `CommissionEmployee`'s `earnings` method directly affects how `BasePlusCommissionEmployees` are paid, because `BasePlusCommissionEmployee`'s `earnings` method calls `CommissionEmployee`'s `earnings` method.
- But changing the `CommissionCompensationModel`'s `earnings` implementation does not affect `BasePlusCommissionCompensationModel`, because these classes are not tightly coupled by inheritance

Rethinking the Employee Hierarchy (5)

Flexibility if Employees Acquire New Capabilities

- Using composition and interfaces, also, is more flexible than the previous class hierarchy for enhancing class `Employee`
- Let's assume we decide to support retirement plans. We could say that every `Employee` *has a* `RetirementPlan` and define interface `RetirementPlan` with a `makeRetirementDeposit` method
- Then, we can provide appropriate implementations for various retirement-plan types

© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

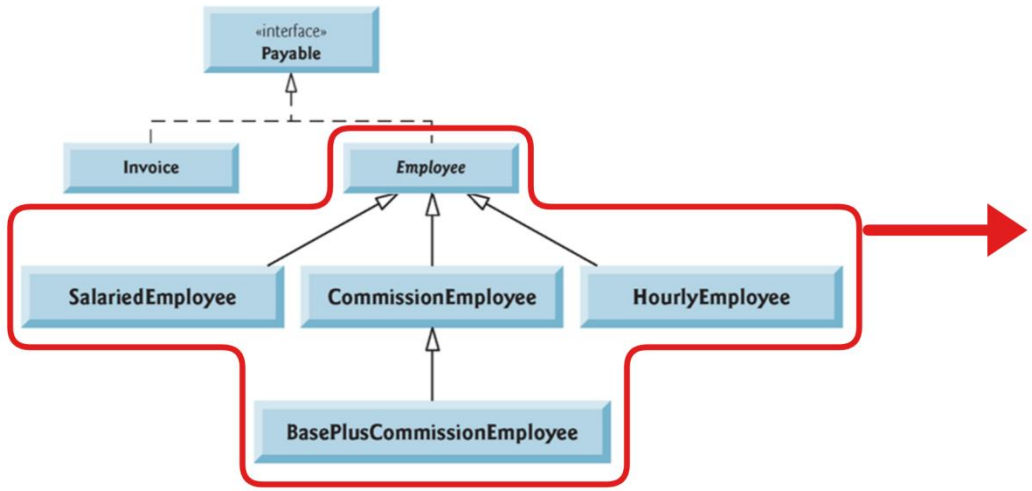
Rethinking the Employee Hierarchy (6)

Flexibility if Employees Are Promoted

- Interface-based composition is more flexible than the previous Employee class hierarchy if an `Employee` gets promoted
- Class `Employee` can provide a `setCompensationModel` method that receives a `CompensationModel` and assigns it to the `Employee`'s composed `CompensationModel` variable
- When an `Employee` gets promoted, you'd simply call `setCompensationModel` to replace the `Employee`'s existing `CompensationModel` object with an appropriate new one
- To promote an employee using 's `Employee` hierarchy, you'd need to change the employee's type by creating a new object of the appropriate class and moving data from the old object into the new one.
- **Exercise**: Reimplement the previous Employee salaries example using interface `CompensationModel` as described in this section.

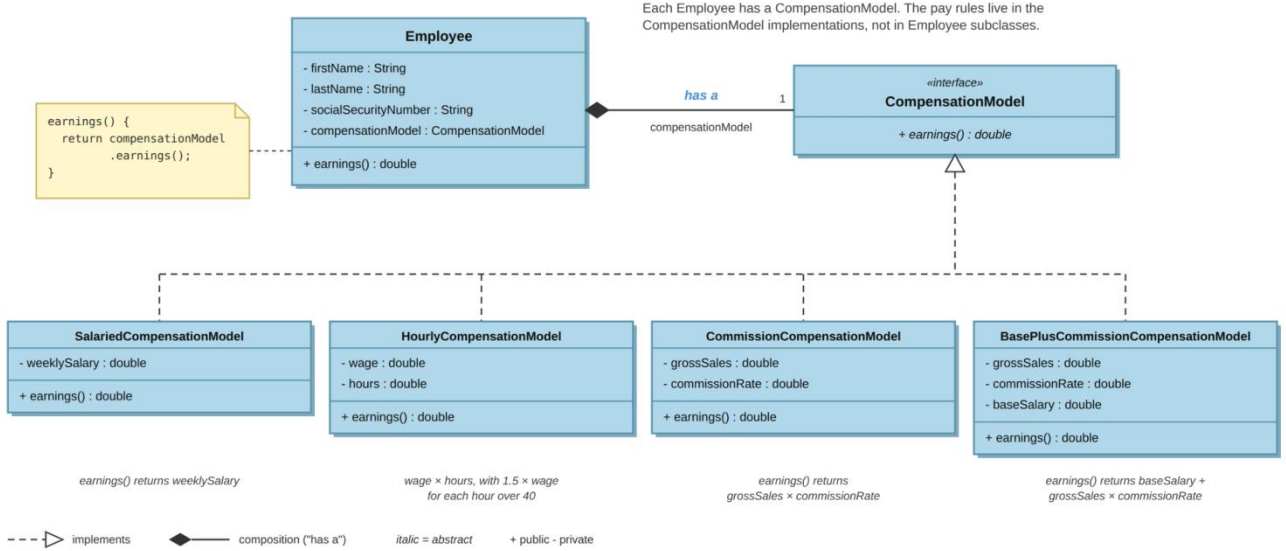
© Copyright 1992-2018 by Pearson Education, Inc. All Rights Reserved.

Original design: inheritance



Employee hierarchy rethought: composition + interface

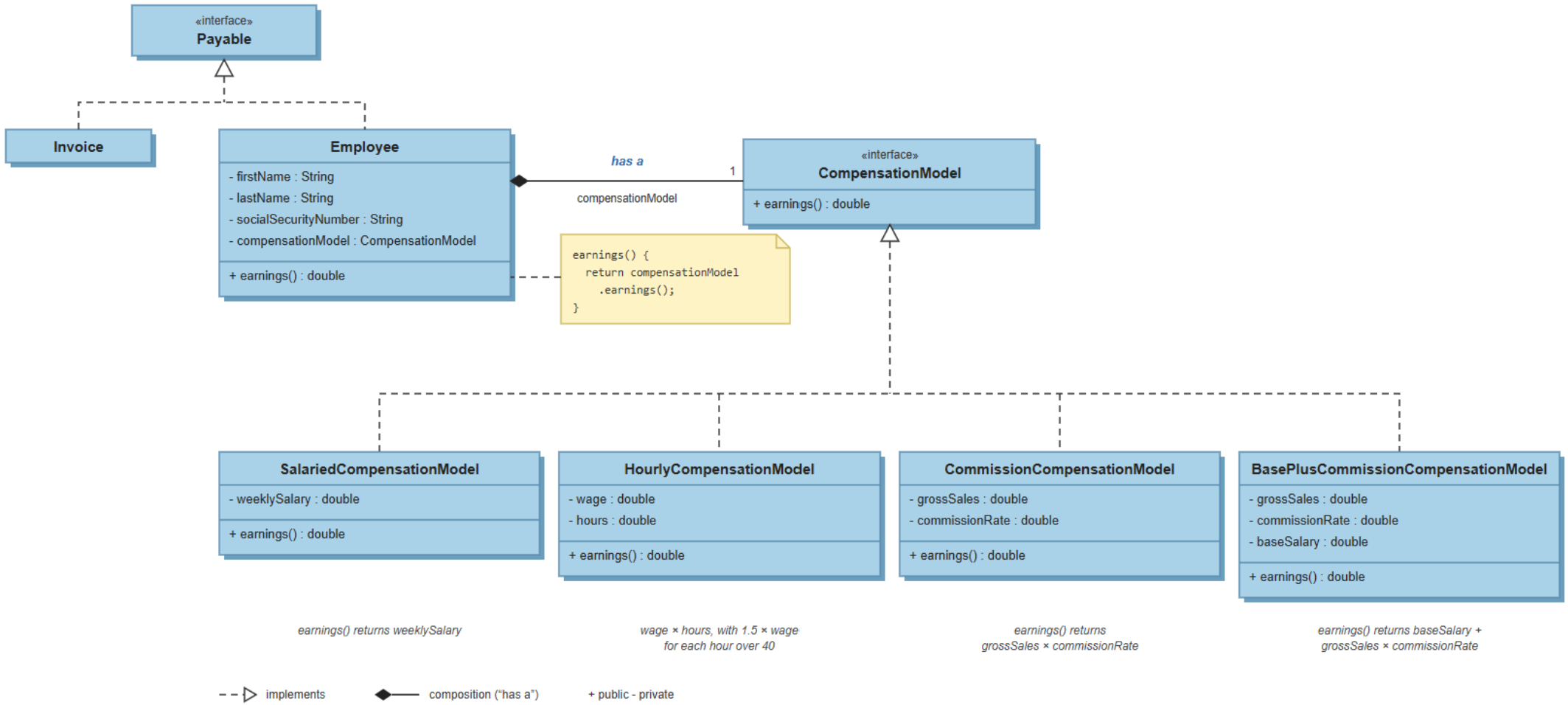
Each Employee has a CompensationModel. The pay rules live in the CompensationModel implementations, not in Employee subclasses.



Example: Chapter04/employeeInterface

Redesigned: composition + interface

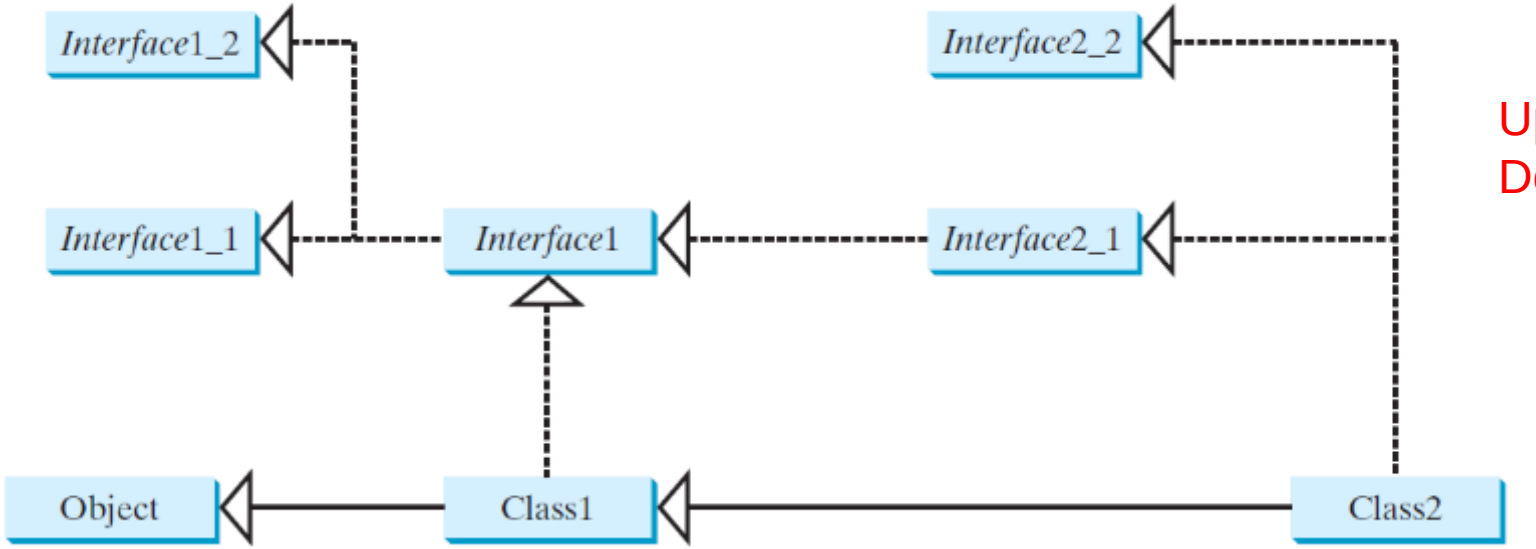
Each Employee has a CompensationModel. The pay rules live in the CompensationModel implementations, not in Employee subclasses.



Solution [15 min]

Interfaces vs. Abstract Classes, cont.

- All classes share a single root, the Object class, but there is no single root for interfaces.
- Like a class, an interface also defines a type. A variable of an interface type can reference any instance of the class that implements the interface. If a class implements an interface, this interface plays the same role as a superclass. You can use an interface as a data type and cast a variable of an interface type to its subclass. and vice versa.

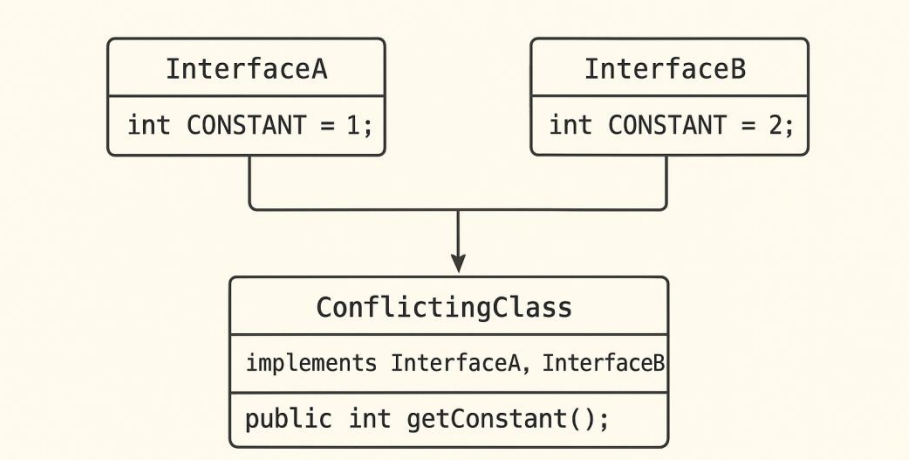


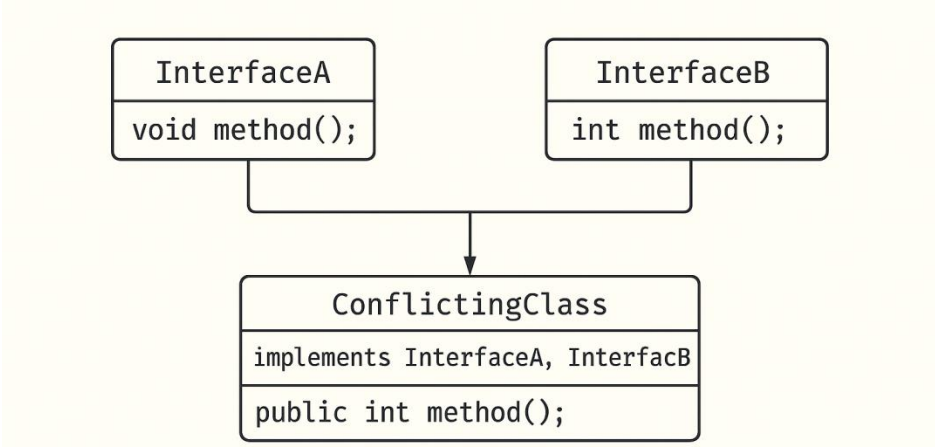
UpCasting ?
DownCasting ?

Suppose that c is an instance of Class2. c is also an instance of Object, Class1, Interface1, Interface1_1, Interface1_2, Interface2_1, and Interface2_2.

Caution: conflict interfaces

In rare occasions, a class may implement two interfaces with conflict information (e.g., two same constants with different values or two methods with same signature but different return type). This type of errors will be detected by the compiler.





Examples: conflict interfaces

Conflicting constants
If both interfaces define constants with the same name, the class cannot refer to the constant directly because it is ambiguous.

```
interface A {
    int VALUE = 10;
}

interface B {
    int VALUE = 20;
}

class Test implements A, B {
    void print() {
        System.out.println(VALUE); //  Error: ambiguous
    }
}
System.out.println(A.VALUE); // 10
System.out.println(B.VALUE); // 20
```

Conflicting methods
If two interfaces declare methods with the **same signature (method name & parameter type) but incompatible return types**, the class cannot implement both

```
interface A {
    int getValue();
}

interface B {
    String getValue();
}

class Test implements A, B {  //  Compiler error
}
```

Whether to use an interface or a class?

Abstract classes and interfaces can both be used to model common features. How do you decide whether to use an interface or a class? In general, a strong **is-a relationship** that clearly describes a parent-child relationship should be modeled using classes. For example, a staff member is a person.

A weak is-a relationship, also known as an is-kind-of relationship, indicates that an object possesses a certain property. A weak is-a relationship can be modeled using interfaces. For example, all strings are comparable, so the String class implements the Comparable interface.

You can also use interfaces to circumvent single inheritance restriction if multiple inheritance is desired. In the case of multiple inheritance, you have to design one as a superclass, and others as interface.

162

Whether to use an interface or a class?

Strong IS A relationship:

- Dog is an Animal
- Car is a Vehicle
- Student is a Person

Weak IS A relationship:

- Sometimes a class is not a specialized version of another class, but simply has a capability or property.
- Example ComissionModel Interface for salary employee, hourly employee etc.
- Example Comparable: A String is **not** a Comparable. Instead, a String is a **kind of object that can be compared**.

```
interface Comparable {
    int compareTo(Object o);
}

class String implements Comparable {
    public int compareTo(Object o) {
        // comparison code
    }
}
```

Designing a Class

(Coherence) A class should describe a single entity, and all the class operations should logically fit together to support a coherent purpose. You can use a class for students, for example, but you should not combine students and staff in the same class, because students and staff have different entities.

```
public class StoreOwner {

private Long id;
private String name;
private String emailId;
private String password;
private String dialCode;
private String phoneNumber;
private LocalDate dateOfBirth;
private String gender;
private Long countryId;
private String bio;
private String profileImage;
private Boolean isActive;
private Boolean isVerified;
private Boolean isDeleted;
private Long updatedBy;
private String coverImage;
private String themeName;
private String storeFrontName;
private String bannerTitle;
private String bannerText;
private String bioTagLine;
private String welcomeText;
private String storeUrl;

private String draftThemeName;
private String draftCoverImage;
private String draftStoreFrontName;
private String draftBannerTitle;
private String draftBannerText;
private String draftBioTagLine;
private String draftWelcomeText;
private String draftBio;
private String draftProfileImage;
private String draftStoreUrl;

private Boolean isPublished;
private LocalDateTime firstPublishedAt;
private LocalDateTime lastPublishedAt;
private Boolean hasDraftChanges;
private Boolean canResellTrip;
private String resellExcludedCountry;

private Boolean termsAccepted;
private LocalDateTime termsAcceptedAt;
private String termsVersion;

private String googleId;
private String authProvider;

private LocalDateTime createdAt;
private LocalDateTime updatedAt;
```

Fig. Example of a non-cohesive class design

Rev: Designing a Class, cont.

(**Separating responsibilities**: Single Responsibility Pattern SOP) A single entity with too many responsibilities can be broken into several classes to separate responsibilities. The classes String, StringBuilder, and StringBuffer all deal with strings, for example, but have different responsibilities.

The String class deals with immutable strings, the StringBuilder class is for creating mutable strings, and the StringBuffer class is similar to StringBuilder except that StringBuffer contains synchronized methods for updating strings.

Exercise: Redesign the class from the previous slide.

h

Solution

```
public class StoreOwner {
    private Long id;
    private String name;
    private String emailId;
    private String password;
    private String dialCode;
    private String phoneNumber;
    private LocalDate dateOfBirth;
    private String gender;
    private Long countryId;
    private String bio;
    private String profileImage;
}
```

Responsibility: Store information about the owner.

```
public class StoreDesign {
    private String themeName;
    private String coverImage;
    private String storeFrontName;
    private String bannerTitle;
    private String bannerText;
    private String bioTagline;
    private String welcomeText;
    private String storeUrl;
}
```

Responsibility: Manage the store's visual/design information

```
public class StorePublication {
    private Boolean isPublished;
    private LocalDateTime firstPublishedAt;
    private LocalDateTime lastPublishedAt;
    private Boolean hasDraftChanges;
}
```

Responsibility: Manage the publishing status of the store.

```
public class Authentication {
    private String googleId;
    private String authProvider;
    private String password;
}
```

Responsibility: Manage authentication-related information.

```
public class TermsAgreement {
    private Boolean termsAccepted;
    private LocalDateTime termsAcceptedAt;
    private String termsVersion;
}
```

Responsibility: Responsibility: Manage acceptance of terms and conditions.

Rev: Designing a Class, cont.

Classes are designed for reuse. Users can incorporate classes in many different combinations, orders, and environments. Therefore, you should design a class that imposes no restrictions on what or when the user can do with it, design the properties to ensure that the user can set properties in any order, with any combination of values, and design methods to function independently of their order of occurrence.

Provide a public no-arg constructor and override the equals method and the toString method defined in the Object class whenever possible.

Rev: Designing a Class, cont.

- Follow **standard Java programming style and naming conventions**. Choose informative names for classes, data fields, and methods.
- Always place the data declaration before the constructor, and place constructors before methods.
- Always provide a constructor and initialize variables to avoid programming errors.
- In java if you do not provide a constructor, the Java compiler generates one:

```
package psu.se411.ch04;                    //  only lowercase letters and digits (no underscores)
public class User {                                  // Class names: UpperCamelCase
    private static final Double SOME_CONST;  // all uppercase letters separated by underscores
    private String name;
    private int age;
    public void doThisNow() {};                 // Methods name: lowerCamaleCase
}
```

⇒Java generates this : public **User**() { super(); }

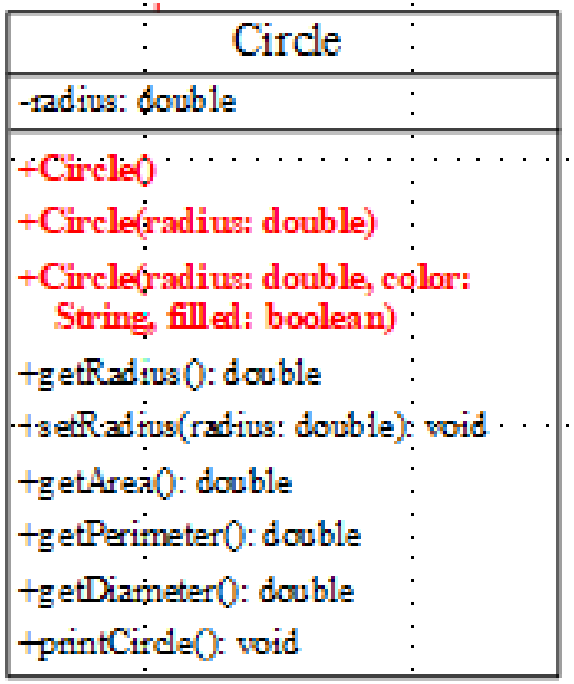
Ref:
https://google.github.io/styleguide/javaguide.html

Rev: Using Visibility Modifiers

- Each class can present two contracts, one for the users of the class (public) and one for the extenders of the class (protected).
- Make the fields private and accessor methods public if they are intended for the users of the class. Make the fields or method protected if they are intended for extenders of the class: Limiting Scope and Exposure is one of the principles of defensive programming.
- The contract for the extenders encompasses the contract for the users.

Immutable Objects and Classes

If the contents of an object cannot be changed once the object is created, the object is called an ***immutable*** *object* and its class is called an *immutable class*. If you delete the set method in the Circle class, the class would be immutable because radius is private and cannot be changed without a set method.



A class with all private data fields and without mutators is not necessarily immutable. For example, the following class **Student** has all private data fields and no mutators, but it is mutable.

Example

```
Public class Circle {

private double radius;

public Circle(double radius) {
    this.radius = radius;
}

public double getRadius() {
    return radius;
}
}
```

```
Circle c = new Circle(5);
```

No setters ≠ automatically immutable.

```
public class Student {
    private String name;
    private int[] grades;

    public Student(String name, int[] grades) {
        this.name = name;
        this.grades = grades;
    }

    public String getName() {
        return name;
    }

    public int[] getGrades() {
        return grades;
    }
}
```

```
int[] g = student.getGrades();
g[0] = 100;
```

To make a class truly immutable
You need to make sure the object's internal state **cannot be changed from outside**, including through references to mutable objects.

No setters ≠ automatically immutable.

```
public class Student {
    private String name;
    private int[] grades;

    public Student(String name, int[] grades) {
        this.name = name;
        this.grades = grades;
    }

    public String getName() {
        return name;
    }

    public int[] getGrades()  {
       return grades.clone();
    }
}
```

To make a class truly immutable
You need to make sure the object's internal state **cannot be changed from outside**, including through references to mutable objects.

private fields and the absence of setters are not sufficient to make a class immutable if it exposes a reference to a mutable object.

Example

```
public class Student {
  private int id;
  private BirthDate birthDate;

  public Student(int ssn,
      int year, int month, int day) {
    id = ssn;
    birthDate = new BirthDate(year, month, day);
  }

  public int getId() {
    return id;
  }

  public BirthDate getBirthDate() {
    return birthDate;
  }
}
```

```
public class BirthDate {
  private int year;
  private int month;
  private int day;

  public BirthDate(int newYear,
      int newMonth, int newDay) {
    year = newYear;
    month = newMonth;
    day = newDay;
  }

  public void setYear(int newYear) {
    year = newYear;
  }
}
```

```
public class Test {
  public static void main(String[] args) {
    Student student = new Student(111223333, 1970, 5, 3);
    BirthDate date = student.getBirthDate();
    date.setYear(2010); // Now the student birth year is changed!
  }
}
```

What Class is Immutable?

- For a class to be immutable, it must mark all data fields private and provide no mutator methods and no accessor methods that would return a reference to a mutable data field object.

- Advantages:

- **Simplicity** - each class is in one state only (simplified reasoning and debugging)
- **Thread Safe** - because the state cannot be changed, no synchronization is required
- Writing in an immutable style can lead to more **robust code**. Imagine if Strings weren't immutable; Any getter methods that return a String would require the implementation to create a defensive copy before the String is returned - otherwise a client may accidentally or maliciously break that state of the object.

String is immutable

```
class Student {
    private String name;

    public String getName() {
        return name;
    }
}
```

Because **String is immutable**, when we return name, the caller cannot change the original String object.

```
String x = student.getName();
x = "Ali";
```

Important interfaces: The Comparable Interface

```
public interface Comparable<E> {
   public int compareTo(E o);
}
```

Since all the numeric wrapper classes and the Character class implement the Comparable interface, the compareTo method is implemented in these classes.

compareTo doc:

Compares this object with the specified object for order. Returns a negative integer, zero, or a positive integer as this object is less than, equal to, or greater than the specified object.

The implementor must ensure `signum(x.compareTo(y)) == -signum(y.compareTo(x))` for all x and y. (This implies that `x.compareTo(y)` must throw an exception if and only if `y.compareTo(x)` throws an exception.)

The implementor must also ensure that the relation is transitive: `(x.compareTo(y) > 0 && y.compareTo(z) > 0)` implies `x.compareTo(z) > 0`.

Finally, the implementor must ensure that `x.compareTo(y)==0` implies that `signum(x.compareTo(z)) == signum(y.compareTo(z))`, for all z.

x<y => y>x

x<y<z => x<z

x==y, if x<z => y<z

Integer and BigInteger Classes

```
public class Integer extends Number
    implements Comparable<Integer> {
  // class body omitted

  @Override
  public int compareTo(Integer o) {
    // Implementation omitted
  }
}
```

```
public class BigInteger extends Number
    implements Comparable<BigInteger> {
  // class body omitted

  @Override
  public int compareTo(BigInteger o) {
    // Implementation omitted
  }
}
```

String and Date Classes

```
public class String extends Object
    implements Comparable<String> {
  // class body omitted

  @Override
  public int compareTo(String o) {
    // Implementation omitted
  }
}
```

```
public class Date extends Object
    implements Comparable<Date> {
  // class body omitted

  @Override
  public int compareTo(Date o) {
    // Implementation omitted
  }
}
```


178

Example:

```
1  System.out.println(new Integer(3).compareTo(new Integer(5)));
2  System.out.println("ABC".compareTo("ABE"));
3  java.util.Date date1 = new java.util.Date(2013, 1, 1);
   java.util.Date date2 = new java.util.Date(2012, 1, 1);
    System.out.println(date1.compareTo(date2));
```


179

Generic `sort` Method

Let **n** be an **Integer** object, **s** be a **String** object, and **d** be a **Date** object. All the following expressions are **true**.

```
n instanceof Integer
n instanceof Object
n instanceof Comparable
```

```
s instanceof String
s instanceof Object
s instanceof Comparable
```

```
d instanceof java.util.Date
d instanceof Object
d instanceof Comparable
```

The java.util.Arrays.sort(array) method requires that the elements in an array are instances of Comparable<E>.

180

Generic `sort` Method: example

```
import java.math.*;

public class SortComparableObjects {

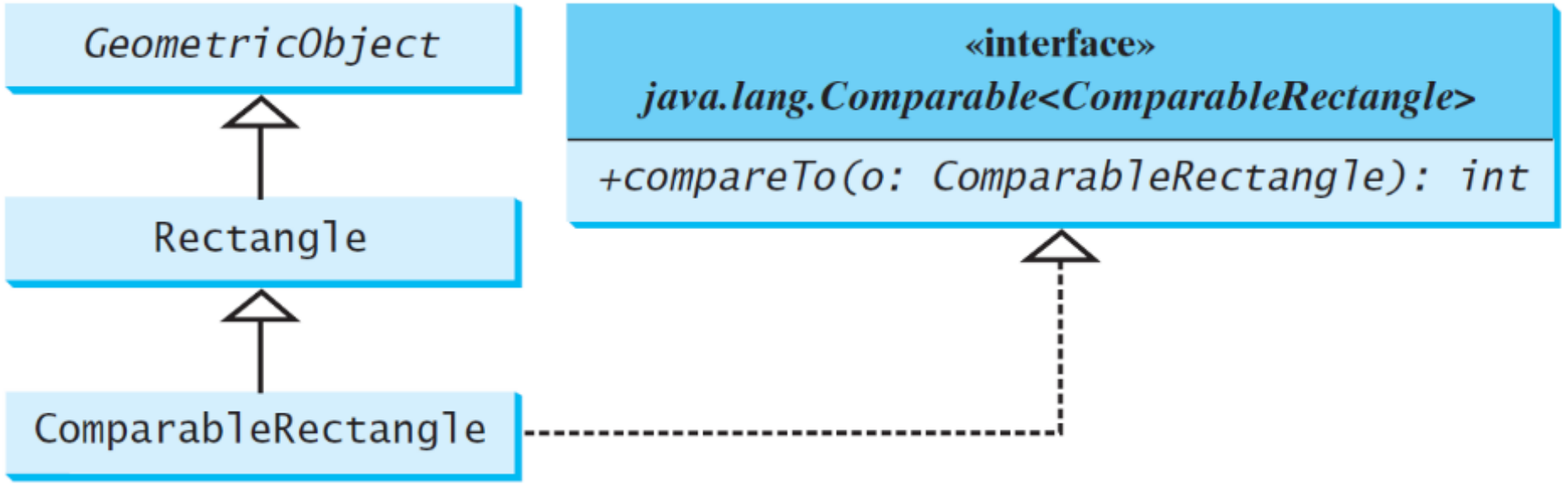
  public static void main(String[] args) {

    String[] cities = {"Savannah", "Boston", "Atlanta", "Tampa"};
    java.util.Arrays.sort(cities);

    for (String city: cities)
      System.out.print(city + " ");

}
```

Defining Classes to Implement Comparable




```
ComparableRectangle[] rectangles = {
        new ComparableRectangle(3.4, 5.4),
        new ComparableRectangle(13.24, 55.4),
        new ComparableRectangle(7.4, 35.4),
        new ComparableRectangle(1.4, 25.4)
};
java.util.Arrays.sort(rectangles);
```

Make class reclangle comparable

```
public class ComparableRectangle extends Rectangle
                                    implements Comparable<ComparableRectangle> {

    public ComparableRectangle(double width, double height) {
      super(width, height);
    }

    @Override
    public int compareTo(ComparableRectangle o) {
      if (getArea() > o.getArea())
        return 1;
      else if (getArea() < o.getArea())
        return -1;
      else
        return 0;
    }

}
```

The `Cloneable` Interfaces

Marker Interface: An empty interface.

A marker interface does not contain constants or methods. It is used to denote that a class possesses certain desirable properties. A class that implements the Cloneable interface is marked cloneable, and its objects can be cloned using the clone() method defined in the Object class.

```
package java.lang;
public interface Cloneable {
}
```


184

Examples

Many classes (e.g., Date and Calendar) in the Java library implement Cloneable. Thus, the instances of these classes can be cloned. For example, the following code

```
Calendar calendar = new GregorianCalendar(2003, 2, 1);
Calendar calendarCopy = (Calendar)calendar.clone();
System.out.println("calendar == calendarCopy is " +
  (calendar == calendarCopy));
System.out.println("calendar.equals(calendarCopy) is " +
  calendar.equals(calendarCopy));
```

displays

calendar == calendarCopy is false
calendar.equals(calendarCopy) is true

185

Implementing Cloneable Interface

To define a custom class that implements the Cloneable interface, the class must override the clone() method in the Object class and make it public (it is protected in the Object class).

Here, class named House implements Cloneable and Comparable.

```
public class House implements Cloneable,
Comparable<House> {
    private int id;
    private double area;
    private java.util.Date whenBuilt;

    public House(int id, double area) {
        this.id = id;
        this.area = area;
        whenBuilt = new java.util.Date();
    }

    @Override
    public House clone() throws CloneNotSupportedException
    {
        House cloned = (House) super.clone();
                return cloned;
    }

    @Override
    public int compareTo(House o) {
        if (area > o.area) return 1;
        else if (area < o.area)  return -1;
        else return 0;
    }
}
```

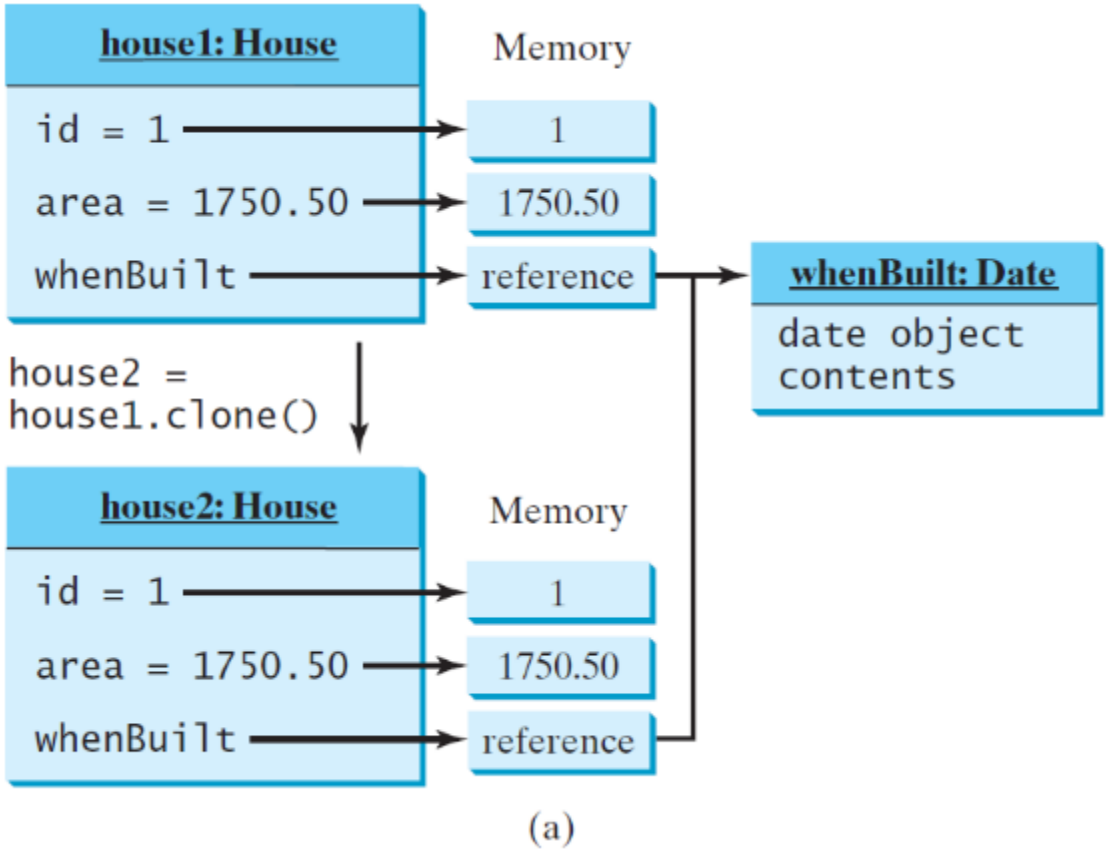
Shallow vs. Deep Copy

The default implementation of Java Object clone() method is using shallow copy.

House house1 = new House(1, 1750.50);

House house2 = house1.clone();

Shallow Copy

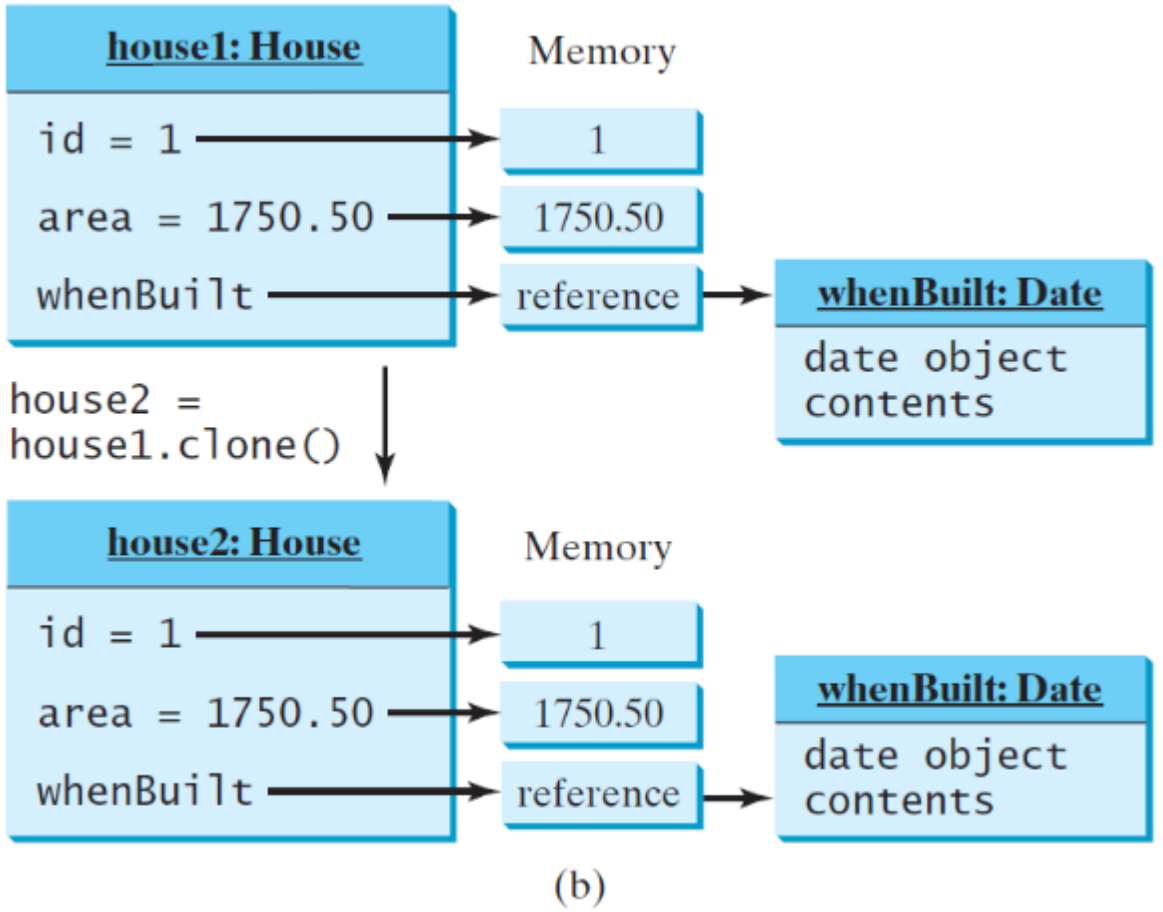


Shallow vs. Deep Copy

House house1 = new House(1, 1750.50);

House house2 = house1.clone();

Deep Copy




```
@Override
public House clone() throws
CloneNotSupportedException {
    House cloned = (House) super.clone();
    cloned.whenBuilt = (Date) whenBuilt.clone();
    return cloned;
}
```


188

When to use Shallow vs. when to use Deep Copy?

```
class Student implements
Cloneable
{ String name; Integer age;
 }
```

```
class Student implements
Cloneable {
String name;
ArrayList<String> courses; }
```

```
class Student implements
Cloneable {
final String name;
 }
```

Scenario	Safe to use `super.clone()`?	Why
Only immutable fields (String, Integer, etc.)	✔ Yes	Shallow copy is enough
Contains mutable objects (List, Map, custom classes)	❌ No	Must deep-clone nested state
Contains final fields	⚠ Depends	`super.clone()` works, but you can't reassign finals

Note: If the class has a final property, after cloning, you cannot reassign it, which is very restrictive and, in most cases makes cloning unuseful.

```
Student s2 = (Student) s1.clone();
```

Exercise: Comparable Employee

- Create a class ComparableEmployee that extends Employee and implements Comparable<ComparableEmployee>.\
- Each employee has:
  name — String
  rank — int
  salary — double
- The compareTo() method should compare employees according to these rules:
  - Employees with a **higher rank** are considered greater.
  - If two employees have the **same rank**, compare their **salary**.
  - Return:
    1 if this employee is greater.
    -1 if this employee is smaller.
    0 if both rank and salary are equal.

```
class ComparableEmployee extends Employee
        implements Comparable<ComparableEmployee> {

    public ComparableEmployee(String name, int rank, double salary)
       {
        super(name, rank, salary);
    }

    @Override
    public int compareTo(ComparableEmployee other) {

        // First compare rank
        if (this.getRank() > other.getRank()) {
            return 1;
        }
        else if (this.getRank() < other.getRank()) {
            return -1;
        }
        // If rank is equal, compare salary
        if (this.getSalary() > other.getSalary()) {
            return 1;
        }
        else if (this.getSalary() <
other.getSalary()) {
            return -1;
        }

        // Rank and salary are equal
        return 0;
    }

}
```